Computer Vision

Interview questions with answers

232 questions (DL03-001 to DL03-232) from the ITW_QUES corpus, grouped by topic.
Question IDs match the RAG files in 02_deep_learning.

Contents

CNN fundamentals	3
Convolution mechanics	6
Pooling and invariance	8
Classic architectures	9
Vision transformers and multimodal models	11
Data augmentation and transfer learning	13
Object detection	16
Segmentation	20
Classical vision, OCR and video	22
Applied design and debugging	26
CNN internals and training details	28
Modern backbones and vision transformers	32
Self-supervised and vision-language representation learning	34
Detection deep dive	37
Segmentation deep dive	40
Classical vision and geometry	42
Video, tracking, pose and 3D	45
Transfer learning, few-shot and domain adaptation	48
Data, robustness and evaluation in vision	52
Applied vision system design and deployment	55
CNN arithmetic and feature extraction	59

CNN fundamentals

DL03-001 What is a convolutional neural network and how does it work?

A CNN is a neural network whose core layers are **convolutions**: small learnable filters (e.g. 3×3) slide over the image and compute a dot product with each local patch, producing a feature map.

- Stacking conv → nonlinearity (ReLU) → occasional downsampling (pooling or stride) builds a hierarchy: edges → textures → parts → objects.
- Each layer sees a larger region of the input (growing receptive field) while sharing weights across positions.
- A head (global pooling + fully connected/softmax layer) turns the final feature maps into class scores.
- Training is standard backpropagation with SGD/Adam on a loss such as cross-entropy.

DL03-002 Describe the architecture of a typical CNN and the role of each kind of layer in it.

Typical block: **Conv** → **BatchNorm** → **ReLU**, repeated, with downsampling every few blocks, then a head.

- **Convolution:** learns local feature detectors; output channels = number of filters.
- **Normalization (BN):** stabilizes activations, allows higher learning rates.
- **Activation (ReLU/GELU):** adds nonlinearity.
- **Pooling / strided conv:** reduces spatial size, increases receptive field, adds small-shift invariance.
- **Residual connections (modern CNNs):** keep gradients flowing in deep stacks.
- **Global average pooling + FC + softmax:** aggregates spatial features into a vector and maps it to class probabilities.
- **Dropout (head):** regularization.

DL03-003 Why are CNNs better suited to image tasks than fully connected networks applied to raw pixels?

- **Parameter efficiency:** a fully connected layer on a 224×224×3 image with 1,000 hidden units needs ~150M weights; a 3×3×3×64 conv needs 1,728.
- **Locality prior:** nearby pixels are correlated; convolutions model local patterns directly.
- **Weight sharing / translation equivariance:** the same detector works anywhere in the image, so the network does not have to relearn a pattern at every position.
- **Hierarchy:** stacked convs compose simple features into complex ones.
- Result: far less overfitting, less data needed and better generalization than an MLP on raw pixels, which ignores spatial structure entirely (permuting pixels would not change what it can learn).

DL03-004 What is the difference between a fully connected layer and a convolutional layer?

- **Fully connected:** every output connects to every input; weights = in × out; no notion of spatial position; input size fixed.
- **Convolutional:** each output connects only to a local K×K×C_in window (sparse connectivity), and the same kernel is reused at every position (weight sharing); weights = K·K·C_in·C_out, independent of image size; output keeps a spatial layout; works on any input size.
- A conv layer is a fully connected layer with a constrained (sparse, shared, Toeplitz-structured) weight matrix. Conversely an FC layer equals a conv whose kernel covers the whole input.

DL03-005 How does a CNN differ from a traditional neural network in computer vision?

A traditional (fully connected) network flattens the image into a vector and learns a separate weight for every pixel-unit pair. A CNN keeps the 2D grid and uses local, shared filters. Consequences:

- CNNs exploit spatial structure and are translation-equivariant; MLPs are not.
- CNNs have orders of magnitude fewer parameters for the same input.

- CNNs learn hierarchical features automatically, which replaced hand-crafted features (SIFT, HOG) plus a shallow classifier.
- CNNs accept variable input sizes (with global pooling); MLPs need a fixed input length.

DL03-006 What is a feature map in a CNN?

A feature map is the 2D output produced by applying **one filter** across the whole input (plus bias and activation). A conv layer with N filters outputs N feature maps stacked into an H×W×N tensor. Each value measures how strongly that filter's pattern is present at that location. Early feature maps respond to edges and colors; deeper ones to textures, parts and whole objects.

DL03-007 What do the filters in a CNN learn, and how do early-layer filters differ from deep-layer filters?

Filters learn the patterns that reduce the loss.

- **Early layers:** low-level, generic detectors that resemble Gabor filters: oriented edges, color blobs, simple gradients. These are similar across datasets and transfer well.
- **Middle layers:** textures, corners, repeated motifs, simple shapes.
- **Deep layers:** large receptive fields; class-specific parts and objects (faces, wheels, text), increasingly invariant to position and pose.

This is why fine-tuning often freezes early layers and retrains deep layers for a new task.

DL03-008 What is parameter sharing in a convolutional layer, and why does it matter?

Parameter sharing means one kernel (one set of weights) is applied at every spatial position, instead of learning separate weights per position.

- **Fewer parameters:** the count depends on kernel size and channels, not image size; this reduces overfitting and memory.
- **Translation equivariance:** a pattern detected in one place is detected anywhere.
- **Better statistical efficiency:** every position provides a training signal for the same weights.

It is the right prior when the same feature can appear anywhere. It is less suitable when position matters (e.g. aligned faces), which is why some models use locally connected layers or positional information.

DL03-009 How does convolution work on grayscale versus RGB inputs, and what determines the shape of the next layer?

A conv kernel always spans **all input channels**.

- **Grayscale (1 channel):** each filter is K×K×1.
- **RGB (3 channels):** each filter is K×K×3; it convolves each channel and **sums** the results plus a bias, giving one 2D map.
- The **next layer's shape** is H_out × W_out × N, where N = number of filters, and H_out = ⌊(H + 2P − K)/S⌋ + 1 (same for W).
- So the next layer's kernels must be K×K×N.

DL03-010 What does the depth of a convolutional layer mean?

The depth of a conv layer is the **number of filters**, which equals the number of output channels (feature maps). Depth sets how many different features the layer can detect, and it becomes the channel depth of the next layer's input. (Network depth, the number of layers, is a different concept.)

DL03-011 What role do fully connected layers play at the end of a classification CNN?

The fully connected head maps the learned feature vector to class scores. It combines features globally ("wheels + windows + road → car"), and its final layer has one output per class, followed by softmax. Older networks (AlexNet, VGG) had large FC stacks holding most of the parameters (~120M of VGG-16's 138M). Modern networks use global average pooling plus a single linear layer, which needs far fewer parameters and overfits less.

DL03-012 Can fully connected layers be replaced by convolutional layers, and what does that buy you at inference time?

Yes. An FC layer on a k×k×C feature map is equivalent to a k×k convolution with C_out filters, and later FC layers become 1×1 convolutions. This gives a **fully convolutional network**:

- It accepts inputs larger than the training size.
- In one forward pass it produces a spatial **map of predictions**, equivalent to sliding the classifier over the image but sharing computation (OverFeat; the basis of FCN segmentation and dense detection).
- Inference on large images is much cheaper than cropping windows and running the network repeatedly.

DL03-013 Is it a good idea to use a CNN to classify one-dimensional signals such as audio or sensor data?

Often yes. **1D convolutions** over time capture local temporal patterns (waveform shapes, sensor motifs) efficiently, with shift equivariance in time.

- **Raw audio:** 1D CNNs (WaveNet-style, dilated convs for long context).
- **Spectrograms:** 2D CNNs treat time-frequency as an image (common in speech and audio classification).
- **Sensor and accelerometer data, ECG:** 1D CNNs or temporal convolutional networks are strong and cheaper than RNNs.
- **Caveats:** very long-range dependencies need dilation, pooling or attention; if the signal is not locally structured or is mostly tabular features, simpler models may do better.

DL03-014 How do CNNs deal with overfitting?

- **Data augmentation:** flips, crops, color jitter, Mixup/CutMix (the most effective lever).
- **Weight decay (L2)** and **dropout** (mainly in the head, or spatial dropout).
- **Batch normalization** (mild regularizing effect).
- **Global average pooling** instead of big FC layers, to reduce parameters.
- **Transfer learning** from pretrained weights when data is small.
- **Early stopping**, label smoothing, stochastic depth.
- Matching model capacity to the amount of data.

DL03-015 How many learnable parameters does a conv layer with 3x3 kernels, 64 input channels and 128 output channels have?

Weights = K × K × C_in × C_out = 3 × 3 × 64 × 128 = **73,728**. Biases = 128. Total = **73,856** parameters. (If followed by BatchNorm, the bias is usually dropped and BN adds 2 × 128 learnable parameters plus 2 × 128 running statistics.)

DL03-016 Where does batch normalization go in a convolutional block, and what does it do for training?

Standard order: **Conv** → **BN** → **ReLU** (in pre-activation ResNets: BN → ReLU → Conv).

- BN normalizes each channel using the batch mean and variance over (N, H, W), then applies a learnable scale γ and shift β per channel.
- The conv bias becomes redundant (BN's β replaces it), so use bias=False.

- **Training effects:** smoother loss landscape, so higher learning rates work; less sensitivity to initialization; faster convergence; mild regularization from batch noise.
- At inference it uses running averages of the statistics, and it can be folded into the preceding conv weights for speed.

Convolution mechanics

DL03-017 What is padding in a CNN and why is it used?

Padding adds pixels (usually zeros) around the border before convolving.

- **Preserves spatial size:** "same" padding with $P = (K - 1)/2$ for stride 1 keeps H×W, so deep networks do not shrink to nothing.
- **Uses border information:** without padding, edge pixels take part in fewer outputs.
- **Types:** zero, reflect, replicate, circular. "Valid" means no padding (the output shrinks by $K - 1$).
- Zero padding can leak absolute-position cues and create border artifacts; reflect padding reduces this in image-to-image tasks.

DL03-018 What is stride, and how does it affect the output size of a convolution layer?

Stride is the step size of the kernel as it slides. Stride S downsamples the output by roughly a factor of S in each dimension: $H_{out} = \lfloor (H + 2P - K)/S \rfloor + 1$. Stride 2 halves the resolution, cuts compute roughly 4× for later layers and enlarges the receptive field. Large strides can skip information and cause aliasing.

DL03-019 Given input size, kernel size, padding and stride, how do you compute the spatial output size of a convolution?

$H_{out} = \lfloor (H_{in} + 2P - K) / S \rfloor + 1$, and the same for width.

- With dilation D, use the effective kernel $K_{eff} = D(K - 1) + 1$.
- Example: 224 input, K = 7, P = 3, S = 2 gives $\lfloor (224 + 6 - 7)/2 \rfloor + 1 = 112$.
- For transposed convolution: $H_{out} = (H_{in} - 1) \cdot S - 2P + K + output_padding$.

DL03-020 Explain the receptive field of a neuron in a CNN and how it grows with depth.

The receptive field (RF) is the region of the input image that can influence a given activation.

- **Recursion:** $RF_l = RF_{(l-1)} + (K_l - 1) \times$ (product of the strides of all previous layers), with $RF_0 = 1$.
- Example: three 3×3 stride-1 convs give RF = 7. A stride-2 layer doubles the growth rate of every later layer.
- RF grows linearly with depth for stride 1 and exponentially with downsampling or dilation.
- The **effective** RF is smaller and roughly Gaussian: central pixels dominate.
- Detection and segmentation need an RF large enough to cover the objects and their context.

DL03-021 Why would you stack many small 3x3 kernels instead of using a few large kernels?

Two stacked 3×3 convs have the RF of one 5×5; three have the RF of one 7×7 (VGG's insight).

- **Fewer parameters:** $3 \times (3^2C^2) = 27C^2$ versus $49C^2$ for a 7×7, about 45% fewer.
- **More nonlinearities:** three ReLUs instead of one, so the function is more expressive.
- **Hardware efficiency:** 3×3 convs are highly optimized (Winograd).
- Trade-off: more layers mean more sequential steps and activation memory.

DL03-022 How do dilated convolutions differ from regular convolutions, and when are they useful?

A dilated (atrous) convolution inserts gaps of $D - 1$ between kernel taps, so a 3×3 kernel with dilation 2 covers a 5×5 area using only 9 weights.

- It grows the receptive field exponentially when stacked, **without downsampling or extra parameters**.

- **Uses:** semantic segmentation (DeepLab keeps high resolution with large context), audio generation (WaveNet), dense prediction.
- **Pitfall:** gridding artifacts when all layers use the same dilation; mix the rates (hybrid dilated convolution).

DL03-023 What does a 1x1 convolution do, and why is it used in architectures like Inception and ResNet bottlenecks?

A 1×1 convolution is a per-pixel linear map across channels, like an FC layer applied at every location.

- **Changes channel count:** reduces it (bottleneck) or expands it cheaply.
- **Mixes channel information** and adds nonlinearity (network-in-network).
- **Inception:** 1×1 reductions before the expensive 3×3 and 5×5 branches cut FLOPs dramatically.
- **ResNet bottleneck:** 1×1 reduce (e.g. 256 → 64), 3×3 at the low width, 1×1 expand back (64 → 256).
- Also used for projection shortcuts and as the pointwise step of depthwise separable convs.

DL03-024 What is a depthwise separable convolution and what are its advantages?

It factorizes a standard convolution into:

1. a **depthwise** convolution: one K×K filter per input channel, giving spatial filtering only, and
2. a **pointwise** 1×1 convolution that mixes channels.

Advantages: about 8–9× fewer FLOPs and parameters for 3×3 kernels with little accuracy loss, which makes it the basis of MobileNet, Xception and EfficientNet. **Drawbacks:** lower arithmetic intensity, so it is memory-bound on GPUs and real speedups are smaller than the FLOP reduction; slightly less expressive per layer.

DL03-025 How do the computation and parameter counts of a depthwise separable convolution compare to a standard convolution?

For an H×W output, kernel K, C_in input and C_out output channels:

- **Standard:** $H \cdot W \cdot K^2 \cdot C_{in} \cdot C_{out}$ multiply-adds; parameters $K^2 \cdot C_{in} \cdot C_{out}$.
- **Depthwise separable:** $H \cdot W \cdot K^2 \cdot C_{in} + H \cdot W \cdot C_{in} \cdot C_{out}$; parameters $K^2 \cdot C_{in} + C_{in} \cdot C_{out}$.
- **Ratio** $= 1/C_{out} + 1/K^2$. For K = 3 and C_out = 256 that is ≈ 0.115, about 8.7× cheaper.

DL03-026 What is a transposed convolution, and why does it produce checkerboard artifacts?

A transposed convolution ("deconvolution") is a learnable upsampling layer: the gradient of a strided convolution with respect to its input. Each input pixel scatters a weighted copy of the kernel into a larger output, and overlapping contributions are summed.

- **Checkerboard artifacts:** when the kernel size is not divisible by the stride (e.g. K = 3, S = 2), some output pixels receive contributions from more input pixels than their neighbors, producing a periodic uneven pattern.
- **Fixes:** use K divisible by S (K = 4, S = 2), or replace it with resize (bilinear or nearest) followed by a normal conv, or use pixel shuffle (sub-pixel convolution).

DL03-027 How would you compute the FLOPs of a conv layer, and why can FLOPs be a poor predictor of real latency?

- **MACs** = H_out · W_out · K_h · K_w · C_in · C_out / groups; **FLOPs ≈ 2 × MACs** (a multiply and an add). Add bias and activation if needed.
- **Why FLOPs mispredict latency:**
  - **Memory bandwidth:** depthwise and 1×1 convs and element-wise ops are memory-bound.
  - **Kernel launch overhead** and the number of layers (depth versus width).
  - **Parallelism and hardware utilization:** small tensors, odd channel counts not multiples of 8 or 16.
  - **Operator support and fusion** on the target (TensorRT, NPUs), data layout conversions, precision (INT8 or FP16 tensor cores).
  - **Pre- and post-processing** (NMS, resizing) are not in the FLOP count.

- Always benchmark on the target device.

Pooling and invariance

DL03-028 What is the purpose of pooling in a CNN, and what are the different types?

Pooling summarizes a local window into one value.

- **Purpose:** downsample (less compute and memory), enlarge the receptive field, add robustness to small translations and distortions, and control overfitting.
- **Types:** max pooling, average pooling, global average or max pooling (whole map to one value), L2 or stochastic pooling, RoI pooling and RoIAlign (detection), spatial pyramid pooling (multi-scale, fixed-length output), adaptive pooling (output size fixed regardless of input).

DL03-029 What is the difference between max pooling and average pooling, and when would you prefer each?

- **Max pooling** keeps the strongest activation: "is the feature present anywhere in this window". It preserves sharp, sparse features and is more invariant to small shifts. Common inside classification backbones.
- **Average pooling** keeps the mean response: smoother, retains information from the whole window, less noisy gradients. Preferred for **global** pooling before the classifier, and when overall intensity or texture matters.
- Max pooling sends gradient only to the argmax position; average pooling spreads it evenly.

DL03-030 Why do classification CNNs use max-pooling?

For classification, what matters is **whether** a discriminative feature occurs, not exactly where. Max pooling acts like a detector "OR" over the window: it keeps the strongest evidence, discards weak responses and background noise, gives invariance to small translations and reduces resolution cheaply. Empirically it outperformed average pooling inside classic CNNs (AlexNet, VGG). Many modern networks replace it with strided convolutions and use global average pooling only at the end.

DL03-031 What makes CNNs translation invariant, and how is that different from translation equivariance?

- **Equivariance:** shifting the input shifts the output by the same amount: f(T(x)) = T(f(x)). Convolution with weight sharing is translation-**equivariant** (up to border effects and stride).
- **Invariance:** the output does not change: f(T(x)) = f(x). CNNs gain approximate translation **invariance** from pooling, striding and finally global pooling, which discard position.
- In practice CNNs are not perfectly invariant: striding and pooling cause aliasing, so one-pixel shifts can change predictions (fixed by anti-aliased blur pooling), and zero padding leaks absolute position.

DL03-032 Are CNNs robust to rotations? What happens to a CNN's predictions if an image is rotated?

No. Standard CNNs are **not rotation-invariant**: convolution is equivariant to translation only. A rotated image activates different filters, so predictions can change or degrade, especially for large rotations or orientation-dependent classes (6 versus 9). **Remedies:**

- rotation augmentation during training (learned approximate invariance);
- test-time augmentation (average over rotations);
- rotation-equivariant architectures (group-equivariant CNNs, harmonic or steerable filters);
- spatial transformer networks that learn to canonicalize orientation.

Some tasks, such as aerial imagery or microscopy, need full rotation invariance; others, such as natural photos, do not.

DL03-033 What is global average pooling and why has it replaced large fully connected heads in many architectures?

GAP averages each final feature map to a single number, turning an H×W×C tensor into a C-dimensional vector, followed by one linear layer.

- **No parameters**, versus tens of millions in VGG-style FC heads, so it overfits less.
- **Any input size:** spatial dimensions are averaged away.
- **Interpretability:** each channel acts as a class-evidence map, which enables CAM and Grad-CAM localization.
- Introduced by Network-in-Network and used by GoogLeNet, ResNet and EfficientNet.

DL03-034 Can you replace pooling layers with strided convolutions, and what are the tradeoffs?

Yes ("all-convolutional net").

- **Pros:** the downsampling is **learned**; it is more expressive; it is common in modern designs (ResNet stems and stages, GAN discriminators where max pooling hurts gradients).
- **Cons:** extra parameters and FLOPs; slightly less built-in invariance; can alias (pooling or a blur before the stride helps).
- Pooling remains a cheap, parameter-free choice when you just need robust downsampling.

DL03-035 What is local response normalization, and why might it be used in a CNN?

Local response normalization (from AlexNet) normalizes each activation by the summed squares of activations in **neighboring channels** at the same position: $b = a / (k + \alpha \Sigma a_j^2)^\beta$. It mimics lateral inhibition, so strongly firing channels suppress their neighbors, which encourages competition and was said to help generalization. It is obsolete: batch normalization (and later layer or group norm) gives much larger, more reliable benefits, and VGG reported LRN did not help.

Classic architectures

DL03-036 What CNN architectures for image classification do you know?

- **Classic:** LeNet-5, AlexNet (ReLU, dropout, GPUs), ZFNet, VGG (stacked 3×3), GoogLeNet/Inception (multi-branch, 1×1 reductions).
- **Residual era:** ResNet, ResNeXt (grouped convs), DenseNet (dense connections), Wide ResNet, SENet (channel attention).
- **Efficient:** MobileNet v1–v3, ShuffleNet, SqueezeNet, EfficientNet (compound scaling), RegNet.
- **Modern:** ConvNeXt (CNN modernized with transformer design choices), NFNet, and transformer backbones (ViT, DeiT, Swin).

DL03-037 What are the common pre-trained CNN models available in popular frameworks?

torchvision, timm, Keras Applications and the Hugging Face Hub provide ImageNet-pretrained ResNet-18/34/50/101/152, ResNeXt, Wide-ResNet, DenseNet, VGG-11/16/19, Inception-v3, GoogLeNet, MobileNet v2/v3, EfficientNet B0–B7 and V2, RegNet, ConvNeXt, ViT, DeiT and Swin. timm alone offers hundreds of variants with different pretraining (ImageNet-21k, CLIP, self-supervised). Detection and segmentation zoos (torchvision, Detectron2, MMDetection, Ultralytics) add Faster/Mask R-CNN, RetinaNet, DETR, YOLO, DeepLab and SAM.

DL03-038 How did the VGG family simplify network design, and what are its main drawbacks?

VGG used a uniform recipe: only 3×3 convolutions with stride 1 and padding 1, 2×2 max pooling with stride 2, channels doubling after each pool (64 → 512), and depth 16–19.

- **Simplifications:** one block design repeated; stacked small kernels give large receptive fields with fewer parameters and more nonlinearities.

- **Drawbacks:** ~138M parameters (most in the FC layers), ~15.5 GFLOPs per 224^2 image, slow and memory-hungry, hard to train deeper without residuals, worse accuracy per FLOP than ResNet or EfficientNet. Still used as a feature extractor (perceptual and style losses).

DL03-039 What problem did Inception modules solve, and how do they use multiple filter sizes?

Inception addressed **what kernel size to use** and how to go deep and wide without exploding compute. Objects appear at different scales, so an Inception module runs **parallel branches** (1×1, 3×3, 5×5 convs and 3×3 max pooling) and concatenates their outputs along channels, letting the network choose the useful scales. **1×1 convolutions before the 3×3 and 5×5 branches** reduce channels, making this affordable. GoogLeNet reached top ImageNet accuracy (2014) with ~6.8M parameters (12× fewer than AlexNet). Later versions factorized 5×5 into two 3×3, and n×n into 1×n plus n×1.

DL03-040 What is the significance of residual networks (ResNets)?

ResNets (2015) made **very deep networks trainable** (152 layers on ImageNet, 1,000+ on CIFAR) by learning residual functions with identity shortcuts. They solved the degradation problem, won ILSVRC 2015 (3.57% top-5 error), and became the default backbone for detection, segmentation and transfer learning. Residual connections are now ubiquitous, including in transformers.

DL03-041 What are skip connections and residual blocks, and why do they make very deep networks trainable?

A residual block computes **y = x + F(x)**, where F is a few conv-BN-ReLU layers and the skip connection adds the input unchanged (a 1×1 projection if shapes differ).

- **Easy identity:** if extra layers are not needed, F can go to zero, so adding depth cannot make the optimum worse.
- **Gradient flow:** $\partial y/\partial x = I + \partial F/\partial x$, so gradients reach early layers directly through the identity path even when $\partial F/\partial x$ is small, which avoids vanishing gradients.
- **Better conditioned loss landscape** and an implicit ensemble of many shallower paths.

DL03-042 Why does accuracy degrade when you simply stack more layers in a plain CNN?

This is the **degradation problem**: deeper plain networks have **higher training error**, not just test error, so it is an optimization issue, not overfitting.

- It is hard for stacked nonlinear layers to learn even the identity mapping.
- Gradients become small or noisy through many layers (vanishing gradients, mostly handled by BN, but optimization stays poor).
- Signal propagation degrades and the loss surface becomes more chaotic.

Residual connections fix this by making the identity the default.

DL03-043 What is a bottleneck block in ResNet and why is it used?

A bottleneck block (ResNet-50 and deeper) uses three layers: **1×1 conv reduces channels** (e.g. 256 → 64), a **3×3 conv** operates at the reduced width, and a **1×1 conv expands back** (64 → 256), with the skip connection around them. The expensive 3×3 runs on 4× fewer channels, so a block costs about the same as a basic two-layer 3×3 block at lower width while allowing much deeper and wider networks. ResNet-50 has ~25.6M parameters and ~4.1 GFLOPs.

DL03-044 What design ideas make MobileNet efficient for mobile devices?

- **MobileNet v1:** depthwise separable convolutions (8–9× fewer FLOPs); a width multiplier α and resolution multiplier ρ to trade accuracy for speed.
- **MobileNet v2:** inverted residual blocks (expand with 1×1, depthwise 3×3, project with 1×1), linear bottlenecks (no ReLU on the narrow output, to avoid losing information), shortcuts between narrow layers.

- **MobileNet v3:** NAS-designed architecture, squeeze-and-excitation, h-swish activation, a cheaper last stage.
- They are designed for low latency and memory on mobile CPUs and NPUs and quantize well to INT8.

DL03-045 What is compound scaling in EfficientNet and why is it better than scaling depth, width or resolution alone?

EfficientNet scales **depth, width and resolution together** with one coefficient φ: depth ∝ α^φ, width ∝ β^φ, resolution ∝ γ^φ, under $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ (each step roughly doubles FLOPs). Grid search on B0 gave α = 1.2, β = 1.1, γ = 1.15. **Why it is better:** the dimensions are interdependent. Higher resolution needs more depth (a bigger receptive field) and more width (finer patterns). Scaling only one saturates quickly. Balanced scaling reached better accuracy per FLOP (B7: 84.3% top-1 with 66M parameters, 8.4× smaller than the previous best).

DL03-046 How would you choose a backbone for a latency-constrained deployment, and why do parameter count, FLOPs and latency disagree?

1. Define the target: the device (GPU, mobile CPU, NPU, Jetson), runtime (TensorRT, CoreML, TFLite), precision (FP16 or INT8), batch size, latency budget (e.g. p95 < 20 ms) and accuracy floor.
2. Shortlist families that fit the hardware (MobileNetV3, EfficientNet-Lite, RegNet, ResNet-18/50, small ConvNeXt or YOLO variants) and **benchmark them on the actual device**, then fine-tune the best on your data.
3. **Why the metrics disagree:**
   - Parameter count measures memory and size, not compute (a 3×3 conv on a large map has few weights but many FLOPs).
   - FLOPs ignore memory traffic: depthwise convs and element-wise ops are bandwidth-bound.
   - Latency depends on parallelism, kernel launches, layer count, operator fusion and support, data layout, and quantization support.
   - So a low-FLOP model can be slower than a "heavier" ResNet on a GPU.

Vision transformers and multimodal models

DL03-047 How can transformers be used for computer vision tasks and for tasks beyond NLP?

Transformers process a sequence of tokens with self-attention, so any modality that can be tokenized works:

- **Vision:** split images into patches (ViT) for classification; DETR for detection; Mask2Former and SegFormer for segmentation; video transformers (TimeSformer, ViViT); Swin as a general backbone.
- **Multimodal:** CLIP aligns image and text; VLMs (LLaVA and others) feed image tokens into an LLM.
- **Beyond NLP:** speech (Whisper), protein structure (AlphaFold), time series, RL (decision transformer), point clouds, graphs.

Their advantage is global context and scalability with data; their cost is quadratic attention and weaker inductive biases.

DL03-048 How does a Vision Transformer turn an image into a sequence of tokens?

1. Split the H×W×C image into non-overlapping P×P patches, giving $N = HW/P^2$ patches (224^2 with P = 16 gives 196).
2. Flatten each patch ($P^2 \cdot C$ values) and apply a learned linear projection to dimension D. This is equivalent to a conv with kernel = stride = P.
3. Prepend a learnable [CLS] token (optional).
4. Add learned 1D position embeddings, since attention is permutation-invariant.
5. Feed the N + 1 tokens to a standard transformer encoder; classify from CLS (or the mean of the patch tokens) with an MLP head.

DL03-049 What inductive biases do CNNs have that Vision Transformers lack, and what is the practical consequence?

CNNs build in **locality** (each unit sees a small neighborhood), **translation equivariance** (weight sharing), and a **hierarchical, multi-scale** structure with pooling. ViTs only have patchification and learned position embeddings; attention is global from layer one, and spatial relations must be learned from data. **Consequence:** with small or medium datasets ViTs underperform CNNs and overfit; with very large datasets (JFT, LAION) or strong pretraining they match or beat CNNs, because fewer built-in assumptions mean a higher ceiling. Hybrid and local-attention designs (Swin, ConvNeXt-style choices) put some biases back.

DL03-050 Why do ViTs typically need more data or stronger regularization than CNNs to train well?

Because they lack locality and translation priors (DL03-049), ViTs must learn these regularities from data, so they have higher sample complexity and overfit small datasets. They also have weaker gradients early in training and less built-in invariance. **Remedies:** pretraining on huge datasets (ImageNet-21k, JFT) or self-supervised pretraining (MAE, DINO); the DeiT recipe of strong augmentation (RandAugment, Mixup, CutMix, random erasing), stochastic depth, label smoothing, long schedules with AdamW and warmup, and distillation from a CNN teacher; or adding convolutional stems or local attention.

DL03-051 How does CLIP enable zero-shot image classification?

CLIP trains an image encoder and a text encoder so that matching image–caption pairs have high cosine similarity in a shared embedding space (400M web pairs, contrastive loss). **Zero-shot classification:**

1. Write each class name as a prompt ("a photo of a {class}"), often averaging several templates.
2. Encode the prompts into text embeddings, which act as classifier weights.
3. Encode the image and compute cosine similarity to each class embedding; softmax gives probabilities.

No task-specific training is needed; new classes are added just by writing text. Accuracy depends on prompt wording and on whether the concept was common in the pretraining data.

DL03-052 How do self-supervised methods like DINO, SimCLR or masked autoencoders learn visual features without labels?

They create a learning signal from the data itself:

- **SimCLR (contrastive):** two random augmentations of the same image are pulled together in embedding space and pushed away from other images in the batch (InfoNCE loss). The network learns augmentation-invariant semantic features.
- **DINO (self-distillation):** a student network matches the output distribution of a momentum (EMA) teacher across different crops (global and local); centering and sharpening the teacher output prevent collapse. It produces features with emergent object segmentation.
- **MAE (masked autoencoding):** mask ~75% of patches and reconstruct their pixels from the visible ones, which forces the model to learn shape and context.

The pretrained encoder is then used for linear probing or fine-tuning on downstream tasks with few labels.

DL03-053 When would a classic CNN classifier beat a vision-language model outright?

- **Narrow, well-defined domain with plenty of labels** (industrial defects, medical scans, specific product SKUs): a supervised CNN learns domain-specific cues that a VLM never saw.
- **Fine-grained distinctions** (bird species, part numbers), counting, precise localization: VLMs are weak at these.
- **Strict latency, cost or edge deployment:** a small CNN runs in milliseconds on-device.
- **Need for calibrated, deterministic, auditable outputs** and a fixed label set.
- **Images unlike web photos** (X-ray, SAR, microscopy), where pretraining coverage is poor.

VLMs win at open-ended, zero-shot, long-tail or language-dependent tasks.

DL03-054 How would you automatically caption an image?

Image captioning is an encoder–decoder problem:

- **Classic:** a CNN encoder plus an LSTM decoder generating words, with **attention** over spatial features (Show, Attend and Tell). Trained with cross-entropy (teacher forcing), then often fine-tuned with RL on CIDEr (self-critical sequence training).
- **Modern:** a vision transformer or CLIP encoder plus a transformer or LLM decoder (BLIP, GIT, CoCa, LLaVA-style VLMs), pretrained on large image–text corpora.
- **Decoding:** beam search or sampling.
- **Evaluation:** BLEU, METEOR, CIDEr, SPICE, CLIPScore, plus human judgment for hallucinated objects.

Data augmentation and transfer learning

DL03-055 What are augmentations and why do we need them?

Augmentations are label-preserving transformations of training images (flips, crops, color changes and so on). They are needed because:

- they enlarge the effective dataset and reduce overfitting;
- they teach invariances you care about (position, scale, lighting) that the architecture does not guarantee;
- they improve robustness to real-world variation (camera, weather, viewpoint);
- they are essential for data-hungry models (ViTs) and for self-supervised contrastive learning, where augmentations define what "similar" means.

DL03-056 What kinds of image augmentations do you know?

- **Geometric:** horizontal or vertical flip, random resized crop, rotation, scaling, translation, shear, affine and perspective warps, elastic deformation.
- **Photometric:** brightness, contrast, saturation, hue jitter, gamma, grayscale, blur, noise, JPEG compression, sharpening.
- **Occlusion and mixing:** Cutout and random erasing, Mixup, CutMix, Mosaic (YOLO), copy-paste (instance segmentation).
- **Learned or automated policies:** AutoAugment, RandAugment, TrivialAugment, AugMix.
- **Domain-specific:** weather and fog simulation, stain normalization (pathology), style transfer, synthetic rendering.

DL03-057 How does data augmentation improve the performance of a vision model, and what techniques can be used?

Augmentation regularizes the model by exposing it to plausible variations of each training image, so it learns features that are invariant to nuisance factors instead of memorizing pixels. Effects: lower variance and overfitting, better out-of-distribution robustness and often better calibration (Mixup). **Techniques:** random resized crop and flip (the baseline), color jitter, RandAugment or AutoAugment, Mixup and CutMix, random erasing, Mosaic and copy-paste for detection, test-time augmentation at inference. Tune strength to model size: large models and ViTs benefit from heavy augmentation, small models from lighter.

DL03-058 Which augmentations would be harmful for a specific task, such as digit recognition or medical imaging, and why?

Augmentation is harmful when it **changes the label** or creates **impossible inputs**:

- **Digits and text:** rotating 180° turns 6 into 9; horizontal flips make letters invalid; heavy shear changes 1 versus 7.
- **Medical imaging:** left–right flips can swap anatomical side (situs, laterality); color jitter can destroy stain-based diagnostic cues in pathology; intensity changes can alter CT Hounsfield units that carry meaning; aggressive crops can remove the lesion.

- **Traffic signs and directions:** flipping "turn left" into "turn right".
- **Fine-grained or counting tasks:** CutMix or crops can remove the discriminative part or change the count.

Rule: only use transformations under which the label is genuinely invariant.

DL03-059 How do Mixup, CutMix and Cutout work, and why do they help generalization?

- **Cutout / random erasing:** blank out a random rectangle. This forces the model to use multiple parts of the object instead of one discriminative patch.
- **Mixup:** blend two images and their labels: $x = \lambda x_1 + (1 - \lambda)x_2$, $y = \lambda y_1 + (1 - \lambda)y_2$, with $\lambda \sim \text{Beta}(\alpha, \alpha)$. This encourages linear behavior between classes, smoother decision boundaries and better calibration.
- **CutMix:** paste a patch from image B into image A; the label is mixed in proportion to the patch area. It keeps natural local statistics (unlike Mixup's ghosting) and combines occlusion robustness with label mixing.

All three act as strong regularizers, reduce overconfidence and improve robustness and localization.

DL03-060 What is transfer learning and how is it used in practice?

Transfer learning reuses a model trained on a large source task (e.g. ImageNet) as the starting point for a target task. **In practice:**

1. Load pretrained weights.
2. Replace the task head (new classifier with the right number of classes).
3. Either freeze the backbone and train only the head (feature extraction), or fine-tune some or all layers with a small learning rate.

It gives faster convergence, much better accuracy with limited data and lower compute. It is the default for almost all practical vision tasks.

DL03-061 What role do pre-trained models play in transfer learning?

Pre-trained models supply **general visual features** learned from millions of images: edges, textures, parts, object shapes. In transfer learning they are either a **fixed feature extractor** (frozen backbone plus a new classifier) or an **initialization** for fine-tuning. They reduce the labeled data needed, speed up convergence, improve generalization and robustness, and act as a strong prior. The closer the pretraining data is to the target domain, the bigger the benefit.

DL03-062 What are feature extractors in the context of transfer learning?

A feature extractor is a pretrained network with its classification head removed, used to map an image to an embedding (e.g. the 2048-d ResNet-50 vector after global pooling, or an intermediate conv map). The backbone is **frozen**, and features are computed once and fed to a lightweight model (logistic regression, SVM, small MLP, k-NN). This is cheap, works with very little data and avoids overfitting, but it cannot adapt the features to the new domain the way fine-tuning can.

DL03-063 How do you implement transfer learning for an image classification task?

1. **Data:** split train, val and test; resize or crop to the backbone's input size (e.g. 224); normalize with the pretraining mean and std; add augmentation.
2. **Model:** load a pretrained backbone (e.g. torchvision ResNet-50) and replace the final layer: model.fc = nn.Linear(2048, num_classes).
3. **Stage 1:** freeze the backbone, train the head (lr ≈ 1e-3) for a few epochs.
4. **Stage 2:** unfreeze some or all layers and fine-tune with a lower lr (1e-4 to 1e-5), optionally with discriminative learning rates; use AdamW or SGD with momentum, cosine schedule, weight decay; keep BN in eval mode if batches are tiny.
5. **Validate** with early stopping; evaluate on the test set; calibrate thresholds if needed.

DL03-064 In what situations is transfer learning recommended, and when is it not?

Recommended:

- small or medium labeled datasets;
- target images similar to the source (natural photos);
- limited compute or time;
- tasks needing general visual features (classification, detection, segmentation).

Less useful or harmful:

- huge in-domain datasets, where training from scratch can match it, though pretraining still speeds convergence;
- very different modalities (spectrograms, SAR, hyperspectral, X-ray), where low-level features may not transfer and domain-specific pretraining is better;
- different input statistics (16-bit, many channels);
- when the source model's biases or license are unacceptable;
- negative transfer, when source and target tasks conflict.

DL03-065 How do you decide how many layers to freeze when fine-tuning a pre-trained CNN?

It depends on **dataset size** and **domain similarity**:

- **Small data, similar domain:** freeze almost everything and train the head (maybe the last block).
- **Small data, different domain:** freeze only early layers (generic edges) and fine-tune middle and late layers carefully, or use features from earlier layers.
- **Large data, similar domain:** fine-tune all layers with a small learning rate.
- **Large data, different domain:** fine-tune everything, or train from scratch with pretrained initialization.

In practice, train the head first, then progressively unfreeze from the top (gradual unfreezing), and pick the depth by validation score.

DL03-066 Why use a smaller learning rate or discriminative learning rates when fine-tuning a pre-trained backbone?

Pretrained weights are already good. A large learning rate makes big updates that **destroy useful features** (catastrophic forgetting), especially early on when the new head is random and sends large, noisy gradients. Discriminative learning rates (e.g. 10× smaller for early layers than late layers, increasing toward the head) reflect that early layers hold generic features needing little change, while later layers and the new head must adapt more. Warmup and a short frozen-backbone phase serve the same purpose.

DL03-067 How does domain shift between ImageNet and your target domain, such as medical or satellite images, affect fine-tuning?

Domain shift (different textures, color statistics, intensity ranges, resolution, scale, orientation, number of channels) means the pretrained features are less relevant:

- low-level filters may still transfer, but high-level ImageNet semantics (dogs, cars) often do not;
- you need to fine-tune more layers, use more labeled target data and larger learning rates for deep layers;
- BatchNorm statistics must be re-estimated on the target domain;
- the gains from ImageNet pretraining shrink. In-domain pretraining (RadImageNet, satellite foundation models, self-supervised pretraining on unlabeled target images) usually works better.
- Watch for input mismatches (grayscale versus RGB, 12/16-bit data, multispectral bands) that need adapted stems.

DL03-068 Given only the CIFAR-10 dataset, how would you build a model that recognizes whether an image belongs to one of its 10 classes or not?

This is **open-set recognition / out-of-distribution detection**: classify the 10 classes and reject everything else.

- **Baselines:** train a normal classifier and reject when the maximum softmax probability (or max logit or energy score) is below a threshold tuned on validation data.
- **Better:** feature-space methods (Mahalanobis distance to class means, k-NN distance in embedding space); ODIN (temperature plus input perturbation); outlier exposure (train with an auxiliary dataset of non-CIFAR images, e.g. Tiny Images or 80M Tiny, pushing them toward uniform predictions); an added "background" class; a one-class or contrastive approach on the in-distribution data.
- **Evaluate** with AUROC, FPR at 95% TPR, using held-out OOD sets (SVHN, LSUN, Textures).

Object detection

DL03-069 What is object detection, and how does it differ from image classification?

Image classification outputs **one label (or label set) per image**. Object detection outputs, for **every object instance**, a **bounding box plus a class label and a confidence**. So detection must localize (regress box coordinates), handle a variable number of objects at different scales, and suppress duplicates (NMS), and it is trained with combined classification and localization losses. It is evaluated with mAP at IoU thresholds rather than accuracy.

DL03-070 How do two-stage detectors such as Faster R-CNN compare to one-stage detectors such as YOLO or SSD in accuracy, speed and use cases?

- **Two-stage (Faster R-CNN):** a region proposal network proposes ~300–2,000 candidate boxes, then RoIAlign plus a second head classifies and refines each box. Higher accuracy, especially for small objects and precise localization; slower (≈ 5–20 FPS); used where accuracy matters (medical, offline analytics, as the base for Mask R-CNN).
- **One-stage (YOLO, SSD, RetinaNet, FCOS):** predict classes and boxes densely over feature-map locations in one pass. Faster (30–200+ FPS), simpler to deploy; historically less accurate because of foreground–background imbalance (fixed by focal loss and better label assignment). Modern YOLOs match two-stage accuracy on many benchmarks. Used for real-time video, edge devices, robotics.

DL03-071 How does the R-CNN family evolve from R-CNN to Fast R-CNN to Faster R-CNN?

- **R-CNN (2014):** ~2,000 selective-search proposals; each crop warped and passed through a CNN separately; SVMs classify and a regressor refines boxes. Very slow (~47 s per image), multi-stage training.
- **Fast R-CNN (2015):** run the CNN **once** on the whole image; **RoI pooling** extracts fixed-size features for each proposal from the shared feature map; one network with softmax classification plus box regression trained jointly with a multi-task loss. ~0.3 s per image (excluding proposals).
- **Faster R-CNN (2015):** replaces selective search with a learned **region proposal network** sharing the backbone features, so the whole detector is end-to-end and near real-time (~5–17 FPS).
- Later: FPN for multi-scale features, Mask R-CNN (masks plus RoIAlign), Cascade R-CNN (progressively higher IoU thresholds).

DL03-072 What is a region proposal network and how does it work?

An RPN is a small fully convolutional network that slides over the backbone feature map. At each position it considers **k anchors** (e.g. 3 scales × 3 aspect ratios = 9) and predicts:

- an **objectness score** for each anchor (object versus background), and
- **box regression offsets** (Δx, Δy, Δw, Δh) refining the anchor.

Training labels anchors with IoU > 0.7 to a ground-truth box as positive and IoU < 0.3 as negative, sampling 256 anchors per image at a 1:1 ratio; the loss is binary cross-entropy plus smooth-L1. At inference the top-scoring

refined anchors go through NMS (IoU 0.7) and the top ~300–2,000 become proposals for the second stage.

DL03-073 What are anchor boxes and how are they assigned to ground-truth objects during training?

Anchors are **predefined reference boxes** of several scales and aspect ratios tiled at every feature-map location; the detector predicts class scores and offsets relative to them. **Assignment during training:**

1. Compute IoU between every anchor and every ground-truth box.
2. An anchor is **positive** if its IoU ≥ a high threshold (e.g. 0.5 in RetinaNet, 0.7 in the RPN) with some ground truth, and is assigned to the box with the highest IoU.
3. For each ground truth, the highest-IoU anchor is also made positive, so every object gets at least one.
4. Anchors below a low threshold (0.4 or 0.3) are **negative** (background); those in between are ignored.
5. Positives get regression targets: $t_x = (x - x_a)/w_a$, $t_y = (y - y_a)/h_a$, $t_w = \log(w/w_a)$, $t_h = \log(h/h_a)$.

Anchor sizes and ratios should match the dataset (k-means on box sizes, as in YOLOv2).

DL03-074 What is non-maximum suppression and why is it needed in object detection?

Detectors output many overlapping boxes for the same object. Non-maximum suppression keeps the best one:

1. Sort boxes by confidence (per class).
2. Take the highest-scoring box and keep it.
3. Remove all remaining boxes whose IoU with it exceeds a threshold (typically 0.5–0.7).
4. Repeat until no boxes remain.

Without NMS you get duplicate detections, which count as false positives and lower precision. **Limitations:** a fixed threshold can delete true detections of heavily overlapping objects (crowds); it is a hand-tuned, sequential, non-differentiable step. Alternatives: soft-NMS, learned NMS, and DETR-style set prediction.

DL03-075 How does the DETR model differ from anchor-based detectors?

DETR (DEtection TRansformer) treats detection as **direct set prediction**:

- A CNN backbone feeds a transformer encoder–decoder; **N learned object queries** (e.g. 100) each output one box plus class (or "no object").
- **Training:** bipartite **Hungarian matching** finds the optimal one-to-one assignment between predictions and ground-truth boxes (cost = class probability + L1 + generalized IoU), then the loss is computed on matched pairs.
- **Differences from anchor-based detectors:** no anchors, no hand-designed assignment rules, **no NMS** (the one-to-one matching discourages duplicates), global reasoning through attention.
- **Downsides of the original:** slow convergence (500 epochs) and weak small-object accuracy. Deformable DETR, DINO-DETR and RT-DETR fix these.

DL03-076 Why is focal loss useful for one-stage detectors, and what problem does it solve?

One-stage detectors evaluate ~100k candidate locations per image, almost all **easy background**. With plain cross-entropy, these easy negatives dominate the loss and gradients, so training is overwhelmed and the classifier learns little about the rare foreground. **Focal loss** (RetinaNet): $FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$, usually $\gamma = 2$ and $\alpha = 0.25$. The factor $(1 - p_t)^\gamma$ down-weights well-classified examples ($p_t \to 1$) by orders of magnitude, so training focuses on hard examples. This let one-stage detectors match two-stage accuracy without hard-negative mining or sampling heuristics.

DL03-077 How do feature pyramid networks help detect objects at different scales?

CNN features are semantically strong but coarse at deep layers, and fine but weak at shallow layers. A **feature pyramid network** builds a multi-scale pyramid with strong semantics at every level:

- **Bottom-up:** the normal backbone (C2–C5, strides 4–32).
- **Top-down:** upsample the deeper map by 2×.

- **Lateral connections:** add a 1×1-projected shallower map to it, then smooth with a 3×3 conv, giving P2–P5 (P6/P7 for large objects).

Objects are predicted on the level matching their size (small objects on high-resolution P2/P3, large on P5). This greatly improves small-object detection at little cost and is used by Faster R-CNN, RetinaNet, Mask R-CNN and YOLO necks (PANet, BiFPN add bottom-up paths).

DL03-078 How would you improve a detector's performance on small objects?

- **Resolution:** larger input images, tiling or slicing inference (SAHI), keeping a stride-4 or stride-8 output level (P2 in FPN).
- **Architecture:** FPN or PAN or BiFPN fusion, dilated convs, attention, less early downsampling.
- **Anchors and assignment:** smaller anchor sizes matching the data, IoU-based or adaptive assignment (ATSS), center sampling, losses less sensitive to tiny boxes (normalized Wasserstein distance).
- **Data:** copy-paste and oversampling of small objects, mosaic, scale augmentation, multi-scale training and testing, fixing missing or noisy small-object labels.
- **Post-processing:** lower NMS thresholds or soft-NMS for dense small objects; tuned confidence thresholds.
- **Evaluate** with AP_small to track progress.

DL03-079 What is intersection over union (IoU) and how is it used in training and evaluating detectors?

IoU = area(A ∩ B) / area(A ∪ B), from 0 (no overlap) to 1 (perfect match).

- **Training:** assigning anchors or predictions to ground truth (positive if IoU > threshold), NMS, and directly as a loss: IoU loss, GIoU, DIoU and CIoU add terms for non-overlapping boxes, center distance and aspect ratio.
- **Evaluation:** a detection is a true positive if IoU ≥ threshold (0.5 for VOC, 0.5:0.95 for COCO) with an unmatched ground truth of the same class; otherwise it is a false positive. These matches drive precision, recall and AP.

DL03-080 How would you evaluate an object detector using IoU and mean average precision?

1. For each class, sort all detections by confidence.
2. Match each detection to an unmatched ground-truth box of that class with IoU ≥ threshold. A match is a TP; otherwise it is an FP (duplicates are FPs). Unmatched ground truths are FNs.
3. Compute cumulative precision and recall down the ranked list, giving a precision–recall curve.
4. **AP** = area under the interpolated PR curve (precision made monotonic; VOC 2007 uses 11 points, COCO uses 101 points).
5. **mAP** = mean AP over classes. COCO also averages over IoU thresholds 0.50:0.05:0.95 and reports AP50, AP75 and AP for small, medium and large objects.

Also check per-class AP, recall at a fixed precision, and speed.

DL03-081 Your detector has high mAP at IoU 0.5 but low mAP at 0.75. What does that tell you and how would you improve it?

The detector **finds the objects** (correct class, rough location) but its **boxes are imprecise**: localization quality is poor, not recognition. **Likely causes:** coarse feature resolution, weak box regression loss, inconsistent or loose annotation boxes, anchor mismatch, aggressive downsampling, small objects, low-IoU positive assignment during training. **Improvements:**

- better regression losses (GIoU, DIoU, CIoU, distribution focal loss);
- cascade refinement (Cascade R-CNN with rising IoU thresholds);
- IoU-aware or quality-aware classification scores (IoU-aware heads, varifocal loss) so NMS keeps the best-localized box;
- higher-resolution features (FPN, P2) and RoIAlign instead of RoIPool;
- stricter positive assignment;

- cleaning box annotations;
- test-time augmentation with weighted box fusion.

DL03-082 How do 3D object detectors working on point clouds differ from 2D image detectors, and why are they computationally heavy?

- **Input:** a point cloud is sparse, irregular and unordered (x, y, z, intensity), not a dense grid; density falls off with distance; there is no color or texture.
- **Output:** 3D boxes with 7+ degrees of freedom (x, y, z, l, w, h, yaw) in metric space, usually in bird's-eye view.
- **Architectures:** point-based (PointNet++, PointRCNN), voxel-based (VoxelNet, SECOND with sparse 3D convs), pillar-based (PointPillars, which flattens to a BEV pseudo-image), range-view, and center-based heads (CenterPoint).
- **Why heavy:** 100k+ points per sweep; 3D voxel grids are huge (cost grows cubically with resolution); neighbor search and grouping for point methods is expensive; the large spatial extent (±100 m) and multi-sweep accumulation add more. Sparse convolutions and pillars are used to cut cost.

DL03-083 How would you design the neural network architecture for multi-camera 3D object detection without lidar?

Goal: predict 3D boxes from N surround-view cameras.

1. **Shared 2D backbone** (ResNet, ConvNeXt or ViT) plus FPN extracts per-camera features.
2. **View transform to bird's-eye view (BEV):**
   - **Lift-splat (LSS, BEVDet):** predict a per-pixel depth distribution, lift features into 3D frustums, and splat them onto a BEV grid using camera intrinsics and extrinsics.
   - **Query-based (BEVFormer, DETR3D, PETR):** BEV or object queries attend to image features by projecting 3D reference points into each camera (deformable or spatial cross-attention).
3. **Temporal fusion:** align previous BEV features with ego-motion (BEVFormer temporal self-attention, BEVDet4D) for velocity and occlusion handling.
4. **BEV encoder plus 3D detection head** (CenterPoint-style heatmaps or DETR-style set prediction with Hungarian matching).

Depth estimation is the key weakness. Supervise depth with lidar during training (BEVDepth), use multi-frame stereo cues, and calibrate cameras precisely.

DL03-084 Why might a self-driving system use lidar, radar and cameras together rather than cameras alone, and where would you fuse them?

Sensors fail differently:

- **Cameras:** rich semantics (lights, signs, lanes, color) and cheap, but poor direct depth and they fail in glare, darkness and fog.
- **Lidar:** precise 3D geometry and distance, works at night; expensive, degraded by rain and snow, no color.
- **Radar:** measures velocity directly (Doppler) and range, works in fog and rain, long range; low resolution and noisy.

Fusing them gives **redundancy** (safety), complementary strengths and robustness to adverse conditions. **Fusion points:**

- **Early (raw or data level):** project lidar points onto images or paint points with image semantics (PointPainting).
- **Mid (feature level):** fuse in a shared BEV feature space (BEVFusion, TransFusion). This is the modern default.
- **Late (object level):** fuse per-sensor detections and tracks with a Kalman filter. Simple and modular but loses information.

Accurate calibration and time synchronization are prerequisites.

DL03-085 How do Google and Apple Maps blur license plates and faces?

They run an automated **detect-then-blur pipeline** on Street View and Look Around imagery:

1. **Detection:** high-recall detectors for faces and license plates (CNN-based, multi-scale, trained on large labeled sets), tuned for recall over precision because missing one is a privacy failure.
2. **Anonymization:** apply strong Gaussian blur or pixelation to the detected regions (irreversible at that resolution) before publishing.
3. **Quality control:** manual review queues and user-reported "request blurring" flows for misses; whole houses or people can be blurred on request.

Challenges: small, distant, occluded or angled faces and plates, false positives (blurring signs), and panorama distortion.

Segmentation

DL03-086 What is image segmentation, and what are its applications?

Image segmentation assigns a label to **every pixel**, partitioning the image into meaningful regions.

- **Semantic:** class per pixel.
- **Instance:** separate mask per object.
- **Panoptic:** both, for things and stuff.

Applications: medical imaging (tumor and organ delineation), autonomous driving (road, lane, drivable area), satellite and aerial mapping (buildings, land cover), photo editing and background removal, AR, robotics grasping, agriculture (crop and weed), industrial inspection, video matting.

DL03-087 What is the difference between semantic segmentation and instance segmentation?

- **Semantic segmentation** labels each pixel with a class but does **not separate instances**: two adjacent people form one "person" region. Usually also covers "stuff" (sky, road).
- **Instance segmentation** detects each object and produces a **separate mask per instance** (person 1, person 2), typically for countable "things" only. It is detection plus masks (Mask R-CNN).
- **Panoptic segmentation** combines both: every pixel gets a class, and thing pixels also get an instance ID.

DL03-088 What segmentation architectures do you know?

- **Semantic:** FCN, SegNet, U-Net (and nnU-Net for medical), DeepLab v1–v3+ (atrous convs, ASPP), PSPNet (pyramid pooling), HRNet (high-resolution throughout), SegFormer (transformer).
- **Instance:** Mask R-CNN, YOLACT, SOLO and SOLOv2, CondInst, PointRend (boundary refinement).
- **Panoptic and universal:** Panoptic FPN, MaskFormer and Mask2Former, OneFormer.
- **Promptable and foundation:** SAM and SAM 2.
- **Real-time:** BiSeNet, Fast-SCNN, DDRNet.

DL03-089 Why do segmentation CNNs typically have an encoder-decoder structure?

Segmentation needs both **what** (semantic context, which needs large receptive fields and deep, downsampled features) and **where** (precise pixel locations, which needs high resolution).

- The **encoder** (a classification backbone) downsamples to capture context and semantics cheaply.
- The **decoder** upsamples back to input resolution (transposed convs or bilinear upsampling plus convs) to produce a dense per-pixel output.
- **Skip connections** bring fine spatial detail from the encoder to the decoder.

Without the decoder, outputs are coarse (stride 16–32) and boundaries are blurry. Without the downsampling encoder, computation at full resolution is prohibitive and context is limited.

DL03-090 How does U-Net use skip connections, and why are they important for localization?

U-Net is a symmetric encoder–decoder ("U" shape). At each decoder level, the upsampled features are **concatenated** with the encoder features of the same resolution (skip connections), followed by convs.

- Downsampling loses precise spatial information; skips reinject **high-resolution edges and textures**, so the decoder can place boundaries accurately while using deep semantic context.
- They improve gradient flow and training with small datasets (U-Net was designed for biomedical images with few labels, plus heavy elastic augmentation).

Concatenation (rather than addition, as in FPN) lets the decoder learn how to combine the two sources.

DL03-091 How does Mask R-CNN extend Faster R-CNN, and what does RoIAlign fix compared to RoIPool?

Mask R-CNN = Faster R-CNN plus a parallel **mask head**: a small FCN that predicts a 28×28 binary mask per RoI for each class. A per-class sigmoid with binary cross-entropy decouples masks from classification. The box branch and RPN stay the same. **RoIAlign fixes RoIPool's quantization.** RoIPool rounds RoI coordinates and bin boundaries to integer feature-map cells, which causes misalignments of up to half a stride (e.g. 16 px), fine for classification but bad for pixel masks. **RoIAlign** keeps floating-point coordinates and **bilinearly interpolates** features at regularly sampled points in each bin, then pools. This gave large mask AP gains (+10–50% relative at strict IoU) and also improved box AP.

DL03-092 How do fully convolutional networks make dense per-pixel predictions from arbitrary-sized input?

An FCN replaces every fully connected layer with convolutions (FC → 1×1 conv), so the network contains only convs, pooling and upsampling. Convolutions work on any spatial size, so an input of any H×W produces a proportionally sized class-score map. Coarse outputs (stride 32) are **upsampled** with learnable transposed convs or bilinear interpolation back to input resolution, and FCN-16s and FCN-8s add **skip connections** from earlier, finer layers to recover detail. Training uses per-pixel cross-entropy on the full image, efficiently sharing computation across all pixels.

DL03-093 Why is plain pixel-wise cross-entropy a poor loss for heavily imbalanced segmentation masks, and what are alternatives such as Dice loss?

With heavy imbalance (e.g. a 1% tumor region), per-pixel CE is dominated by the many easy background pixels. Predicting all background already gives a low loss and 99% accuracy, so the model under-segments small structures. **Alternatives:**

- **Dice loss:** $1 - 2\Sigma(p \cdot g)/(\Sigma p + \Sigma g)$. It optimizes overlap directly, is normalized by region size and is insensitive to background count.
- **Tversky loss** (weights FP versus FN; good for recall), **focal loss** (down-weights easy pixels), **Lovász-softmax** (a surrogate for IoU), **boundary losses**.
- **Weighted CE** (inverse class frequency).
- **Combo Dice + CE:** common default; CE gives stable gradients, Dice handles imbalance.

Patch sampling around foreground also helps.

DL03-094 How do you evaluate a segmentation model with IoU, mean IoU and Dice coefficient?

For prediction mask P and ground truth G, per class:

- **IoU (Jaccard)** = $|P \cap G| / |P \cup G| = TP / (TP + FP + FN)$.
- **Mean IoU** = average IoU over classes, usually computed from a confusion matrix accumulated over the whole dataset (not averaged per image); background included or not per convention. Frequency-weighted IoU weights by class frequency.
- **Dice (F1)** = $2|P \cap G| / (|P| + |G|) = 2TP / (2TP + FP + FN)$. Dice = $2 \cdot IoU / (1 + IoU)$, so it is always ≥ IoU; common in medical imaging, often reported per case.

- **Complements:** pixel accuracy (misleading under imbalance), boundary metrics (Boundary F1, Hausdorff distance HD95) for contour quality, and AP-based metrics for instance segmentation.

Classical vision, OCR and video

DL03-095 What is computer vision, and how does it relate to human vision?

Computer vision is the field of making computers **extract meaning from images and video**: recognizing objects, reading text, estimating depth and motion, understanding scenes. It is loosely inspired by human vision: the visual cortex processes information hierarchically, from edges (V1 simple cells, which motivated Gabor-like filters and CNNs) to shapes and objects. But humans use rich priors, attention, 3D understanding, context and learning from few examples, with strong invariance; machines rely on statistical learning from data, can be fooled by adversarial noise and textures, and can exceed humans on narrow tasks while lacking general understanding.

DL03-096 What is the difference between image processing and computer vision?

- **Image processing:** image in, **image out**. It transforms or enhances pixels: denoising, sharpening, contrast adjustment, filtering, resizing, compression, color correction. No semantic understanding is required.
- **Computer vision:** image in, **understanding out**: labels, boxes, masks, depth, 3D structure, descriptions.

Image processing is often a preprocessing step inside CV pipelines; the boundary blurs with learned methods (super-resolution, inpainting).

DL03-097 What are the common image preprocessing steps in a computer vision pipeline?

- **Decode and convert color space:** BGR → RGB (OpenCV loads BGR), handle grayscale and alpha, correct EXIF orientation.
- **Resize:** aspect-preserving resize plus crop or letterbox padding to the model input size.
- **Normalize:** scale to [0, 1] and standardize with the per-channel mean and std used in pretraining; convert to CHW float.
- **Optional:** denoising, histogram equalization or CLAHE, deskewing (OCR), cropping regions of interest, stain or intensity normalization (medical).
- **Training only:** augmentation.
- Keep training and serving preprocessing **identical**, to avoid training–serving skew.

DL03-098 How does image resizing affect model performance?

- **Downscaling** loses fine detail (small objects, text, textures) and can alias; small objects may vanish entirely.
- **Upscaling** adds no information but increases compute.
- **Aspect-ratio distortion** (squashing) changes shapes the model learned; letterboxing or cropping avoids this.
- **Train/test resolution mismatch** shifts the apparent object size and receptive-field statistics, which hurts accuracy (see FixRes).
- **Compute** scales roughly with H×W, so resolution is a key accuracy–latency knob (EfficientNet scales it deliberately).
- **Interpolation method** (bilinear versus area versus nearest; antialiasing) must match between training and serving.

DL03-099 What techniques can reduce noise in an image?

- **Linear filters:** mean or box filter, **Gaussian blur** (good for Gaussian noise but blurs edges).
- **Nonlinear: median filter** (excellent for salt-and-pepper noise, preserves edges), **bilateral filter** (edge-preserving: weights by spatial and intensity distance).
- **Non-local means** and **BM3D** (exploit repeated patches).
- **Frequency domain:** low-pass, Wiener filtering, wavelet thresholding.
- **Temporal averaging** of multiple frames (video, burst photography).

- **Learned:** denoising CNNs (DnCNN), denoising autoencoders, diffusion-based restoration.
- Choose based on the noise type (Gaussian, Poisson, impulse) and how much edge detail must be kept.

DL03-100 How does edge detection work in image analysis, for example with Sobel or Canny?

Edges are locations of strong intensity change, found via image gradients.

- **Sobel:** convolve with 3×3 kernels Gx = [[−1,0,1],[−2,0,2],[−1,0,1]] and Gy (its transpose). Gradient magnitude = $\sqrt{(Gx^2 + Gy^2)}$, direction = atan2(Gy, Gx). Thresholding the magnitude gives edges. Simple, but edges are thick and noise-sensitive.
- **Canny:**
  1. Gaussian smoothing.
  2. Sobel gradients (magnitude and direction).
  3. **Non-maximum suppression** along the gradient direction, to thin edges to 1 px.
  4. **Double threshold** (strong versus weak edges).
  5. **Hysteresis**: keep weak edges only if connected to strong ones.

  The result is thin, continuous, well-localized edges with good noise robustness.
- Others: Laplacian of Gaussian (zero crossings), Prewitt, Scharr, learned edge detectors (HED).

DL03-101 What are color spaces and why do they matter in image processing?

A color space is a coordinate system for representing color:

- **RGB:** device-oriented, channels correlated with brightness.
- **HSV/HSL:** separates hue, saturation and value; good for color thresholding and segmentation, robust to lighting changes.
- **YCbCr/YUV:** luma plus chroma; used in compression, broadcasting and skin detection.
- **CIELAB:** perceptually uniform (Euclidean distance ≈ perceived difference); good for color comparison and transfer.
- **Grayscale:** intensity only.

They matter because the right space makes a task easier (illumination-invariant features, color-based segmentation), because pipelines must match (OpenCV BGR versus RGB bugs), and because color-related augmentations operate in specific spaces.

DL03-102 What is feature extraction in computer vision?

Feature extraction transforms raw pixels into a compact, informative representation that downstream tasks can use.

- **Hand-crafted:** edges, corners (Harris), keypoints and descriptors (SIFT, SURF, ORB), HOG, LBP, color histograms, Gabor filters.
- **Learned:** activations or embeddings from CNNs and ViTs, which are task-adaptive and far more powerful.

Good features are discriminative, robust (invariant to nuisance factors such as lighting, scale and rotation) and compact.

DL03-103 What are feature descriptors, and why are they important in computer vision?

A descriptor is a **vector summarizing the local appearance around a keypoint** (or a whole image) so that the same physical point can be recognized in different images. Examples: SIFT and SURF (gradient histograms, float), ORB, BRIEF and BRISK (binary, Hamming distance), HOG, learned descriptors (SuperPoint, HardNet). They matter because matching descriptors enables image stitching, 3D reconstruction (structure from motion, SLAM), object recognition, image retrieval, tracking and registration. Good descriptors are distinctive and invariant to scale, rotation, illumination and viewpoint changes.

DL03-104 Explain the SIFT algorithm and the invariances it provides.

SIFT (Scale-Invariant Feature Transform, Lowe 2004):

1. **Scale-space extrema:** build a Gaussian pyramid, compute Difference-of-Gaussians, and find local extrema across space and scale.
2. **Keypoint refinement:** subpixel and subscale localization (Taylor fit); reject low-contrast points and edge responses (Hessian ratio).
3. **Orientation assignment:** histogram of gradient orientations around the keypoint; the dominant orientation(s) define the keypoint's angle.
4. **Descriptor:** a 16×16 window rotated to that orientation, split into 4×4 cells, each with an 8-bin orientation histogram, giving a **128-d** vector, normalized, clipped at 0.2 and renormalized.

Invariances: scale, in-plane rotation, illumination (gradients plus normalization), partial robustness to affine and viewpoint change and noise. Matching uses nearest neighbor with Lowe's **ratio test** ($d1/d2 < 0.8$).

DL03-105 How would you remove outliers when estimating a flat plane from noisy samples?

Use **RANSAC**:

1. Randomly sample the minimal set (3 points) and fit a plane.
2. Count inliers: points with distance to the plane below a threshold τ.
3. Repeat N iterations, where $N = \log(1 - p) / \log(1 - w^3)$ for success probability p and inlier ratio w.
4. Keep the plane with the most inliers.
5. Refit the plane with least squares (SVD or PCA on the inliers: the normal is the smallest singular vector).

Alternatives: M-estimators (Huber or Tukey robust least squares), MSAC and MLESAC (score by residuals), PROSAC, LO-RANSAC. Choose τ from the sensor noise level.

DL03-106 How does image registration work, and how do sparse and dense optical flow compare?

Registration aligns two images (or volumes) into a common coordinate frame: choose a transformation model (rigid, affine, homography, deformable), a similarity measure (feature matches, sum of squared differences, normalized cross-correlation, mutual information for multimodal medical images) and an optimizer. Feature-based: detect and match keypoints, then fit the transform with RANSAC. Intensity-based: optimize the transform parameters directly. Deep methods: VoxelMorph and learned matchers.

Optical flow estimates per-pixel motion between frames under brightness constancy ($I_x \cdot u + I_y \cdot v + I_t = 0$).

- **Sparse (Lucas–Kanade):** flow only at selected keypoints, assuming constant flow in a small window; fast, good for tracking features; often pyramidal for large motion.
- **Dense (Horn–Schunck, Farnebäck, deep RAFT):** a flow vector for every pixel with smoothness regularization; richer (segmentation by motion, frame interpolation, action recognition) but more expensive and error-prone in textureless or occluded regions.

DL03-107 What is OCR and what are its main applications?

OCR (optical character recognition) converts images of text (printed or handwritten, scanned or in natural scenes) into machine-readable text. **Applications:** document digitization and search, invoice and receipt and form processing (key-value extraction), ID and passport reading, license plate recognition, check processing, translation apps, accessibility (screen readers for images), postal sorting, scene-text understanding for robots and maps, digitizing historical archives.

DL03-108 How would you build a modern OCR pipeline that handles text detection and text recognition?

1. **Preprocessing:** deskew, denoise, binarize or enhance contrast, correct perspective (documents), normalize resolution (DPI).
2. **Text detection:** find text regions or lines with segmentation-based detectors (DBNet, CRAFT, EAST, PSENet), which handle arbitrary orientation and curved text.
3. **Rectification:** crop and deskew each region; a thin-plate-spline spatial transformer for curved text.
4. **Text recognition:** a CNN or ViT encoder plus a sequence decoder. Options: CRNN (CNN + BiLSTM + **CTC loss**, no character-level alignment needed), attention or transformer decoders (TrOCR, PARSeq), language-model rescoring.
5. **Post-processing:** dictionary or language-model correction, layout analysis (reading order, tables), key-information extraction (LayoutLM, Donut, or VLMs).
6. **Train** on synthetic text (SynthText) plus real data; evaluate with character and word error rates (CER, WER) and detection F1.

DL03-109 How does multi-object tracking work with Kalman filters or deep association methods like DeepSORT?

Tracking-by-detection:

1. **Detect** objects in each frame.
2. **Predict** each existing track's next state with a **Kalman filter** (constant-velocity model on box position and size), which gives a predicted box and uncertainty.
3. **Associate** detections to tracks by solving an assignment problem (**Hungarian algorithm**) on a cost matrix:
   - **SORT:** IoU between predicted and detected boxes.
   - **DeepSORT:** adds an **appearance embedding** (re-ID CNN, cosine distance) and Mahalanobis distance gating, so it can re-identify objects after occlusion; a matching cascade prioritizes recently seen tracks.
4. **Update** matched tracks with the Kalman correction step. Create new tracks for unmatched detections (tentative until confirmed over several frames) and delete tracks unmatched for too long.

Modern variants: ByteTrack (also associates low-score detections), OC-SORT, BoT-SORT, and end-to-end trackers (MOTR).

DL03-110 What are the challenges in real-time object tracking?

- **Occlusion** and objects leaving and re-entering, which cause identity switches.
- **Appearance changes:** pose, lighting, scale, motion blur, similar-looking objects (crowds, uniforms).
- **Fast or erratic motion** and camera motion that break constant-velocity assumptions.
- **Detector errors:** missed and false detections propagate into tracks.
- **Latency and compute budget:** detection plus re-ID embedding per frame at 30+ FPS on edge hardware; handling many objects.
- **Long-term re-identification** across cameras.
- **Initialization and termination** of tracks.
- **Drift** and accumulated error in single-object trackers.
- **Varying frame rates**, dropped frames, network delays.

DL03-111 How would you preprocess video for a model, including frame sampling, resizing and normalization?

- **Decode** efficiently (hardware decoders, PyAV, decord); handle variable frame rate and rotation metadata.
- **Frame sampling:** uniform sampling of T frames across the clip (TSN-style segments) or dense consecutive frames with a stride (e.g. 16 or 32 frames at stride 2–4) for 3D CNNs; resample to a fixed FPS; skip near-duplicate frames for efficiency.

- **Spatial processing:** resize the short side (e.g. 256), then center or random crop (224); apply the **same** crop, flip and color jitter to all frames in a clip so motion stays consistent.
- **Normalize** with per-channel mean and std; arrange as (C, T, H, W) for 3D CNNs or (T, C, H, W).
- **Long videos:** sliding windows or multiple clips at inference, with scores averaged.
- Optionally precompute optical flow (two-stream) or cache features.

Applied design and debugging

DL03-112 What are the steps to build an image recognition system?

1. **Define** the task, classes, metric and latency or deployment constraints.
2. **Collect and label data:** cover real-world variation (lighting, devices, viewpoints); write labeling guidelines and quality checks.
3. **Split** into train, val and test without leakage (group by source or scene).
4. **Preprocess and augment.**
5. **Baseline:** fine-tune a pretrained CNN or ViT; establish metrics.
6. **Iterate:** error analysis (confusion matrix, failure cases), more or cleaner data, hyperparameter tuning, architecture changes.
7. **Evaluate:** held-out test, robustness and calibration checks, per-slice metrics.
8. **Optimize and deploy:** quantization, ONNX or TensorRT, serving API or edge.
9. **Monitor:** drift, accuracy on sampled production data; retrain loop.

DL03-113 Describe how you would implement an image classification system end to end.

- **Problem:** N classes, accuracy and F1 target, inference budget.
- **Data:** gather and label images; deduplicate; stratified (and group-aware) split; check class balance.
- **Pipeline:** a dataset class with decode, aspect-aware resize and crop, normalization, and training augmentation (random resized crop, flip, color jitter, RandAugment, Mixup or CutMix).
- **Model:** pretrained ResNet-50, EfficientNet or ConvNeXt with a new linear head; AdamW, cosine learning rate with warmup, label smoothing; mixed precision; early stopping on val loss and F1.
- **Evaluate:** accuracy, macro-F1, confusion matrix, calibration (ECE; temperature scaling), error analysis and slice metrics.
- **Deploy:** export to ONNX or TorchScript; quantize to FP16 or INT8 and verify accuracy; serve behind an API with batching; identical preprocessing.
- **Operate:** logging, drift monitoring, human review of low-confidence cases, periodic retraining with newly labeled data.

DL03-114 How would you create a model to recognize whether an image is a triangle, a circle or a square?

Simple, controlled problem, so start simple:

- **Classical:** grayscale, threshold, find contours, then approxPolyDP to count vertices (3 = triangle, 4 = square, many = circle), plus circularity $4\pi A/P^2 \approx 1$ for circles and aspect ratio ≈ 1 for squares. Fast and interpretable, no training.
- **Learning-based:** generate a **synthetic dataset** (random size, position, rotation, color, line width, noise, backgrounds) and train a small CNN (a few conv layers plus GAP plus a 3-way softmax) with rotation and scale augmentation, since shape identity is invariant to them. Add an "other" class if non-shapes can appear.
- **Evaluate** on real or hand-drawn shapes to check the synthetic-to-real gap.

DL03-115 What makes object recognition hard under varied lighting, viewpoints and orientations?

- **Lighting:** shadows, highlights, low light (noise), color casts change pixel values drastically, though the object is the same.
- **Viewpoint and pose:** 3D objects look very different from different angles; parts self-occlude.
- **Orientation and scale:** CNNs are not rotation-invariant and have limited scale robustness.
- **Intra-class variation** (many chair designs) versus **inter-class similarity** (husky versus wolf).
- **Occlusion, clutter, deformation**, motion blur, low resolution.
- **Dataset bias:** training data covers a narrow slice of conditions (e.g. daytime, canonical views).

Mitigations: diverse data, augmentation (photometric and geometric), multi-view and 3D-aware models, normalization (histogram equalization), and large pretrained models.

DL03-116 Your image classifier achieves 99% on validation but fails in production. What are the likely causes and how would you debug it?

Likely causes:

- **Distribution shift:** different cameras, lighting, resolution, compression, user behavior.
- **Data leakage** inflated validation: near-duplicates, the same subjects or scenes in train and val.
- **Training–serving skew:** different preprocessing (BGR/RGB, resize method, normalization, EXIF rotation, color range).
- **Label or class-prior shift**, new classes, open-set inputs.
- **Shortcut learning** (watermarks, backgrounds).
- An unrepresentative validation set.

Debugging:

1. Collect and label a sample of production images; measure real accuracy and the confusion matrix.
2. Compare input statistics and embeddings between training and production (drift tests, adversarial validation).
3. Replay production images through the training pipeline to find preprocessing bugs.
4. Re-split validation by source or time and deduplicate.
5. Inspect failures and Grad-CAM for shortcuts.
6. Fix by adding production-like data, augmentation, aligned preprocessing, an OOD rejection option and monitoring.

DL03-117 How would you handle a severely class-imbalanced image classification dataset?

- **Measure correctly:** per-class recall and precision, macro-F1, balanced accuracy, PR curves, not overall accuracy.
- **Data level:** collect or augment more minority examples; oversample minority classes (class-balanced sampler) or undersample the majority; synthetic data or GAN and diffusion augmentation, used carefully.
- **Loss level:** class-weighted cross-entropy, focal loss, class-balanced loss (effective number of samples), logit adjustment or balanced softmax, label-distribution-aware margin (LDAM).
- **Training schemes:** two-stage decoupling (learn the representation with instance sampling, then retrain the classifier with balanced sampling), pretrained backbones, metric learning for rare classes.
- **Decision level:** per-class threshold tuning, calibration, prior correction.

DL03-118 How would you build a vision model with very few labeled images?

- **Transfer learning:** a strong pretrained backbone (ImageNet, CLIP, DINOv2) used as a frozen feature extractor plus a linear probe or k-NN, then light fine-tuning.
- **Zero- and few-shot:** CLIP zero-shot prompts; prototypical networks or a cosine classifier on embeddings.
- **Self-supervised pretraining** on your unlabeled images (SimCLR, DINO, MAE) before fine-tuning.

- **Semi-supervised learning:** pseudo-labeling and consistency regularization (FixMatch, Mean Teacher) using unlabeled data.
- **Heavy augmentation** and synthetic data.
- **Active learning** to label the most informative images.
- **Regularization:** a small model or head, strong weight decay, early stopping; cross-validation for reliable estimates.

DL03-119 How would you quantize, prune or distill a CNN for deployment on an edge device, and what accuracy risks do you watch for?

- **Quantization:** post-training INT8 with a representative calibration set (per-channel weights, per-tensor activations); if accuracy drops, quantization-aware training. FP16 is often near-lossless.
- **Pruning:** structured channel or filter pruning (real speedups on hardware) by magnitude or BN-scale criteria, then fine-tuning; unstructured sparsity needs sparse-kernel support.
- **Distillation:** train a small student (MobileNet or EfficientNet-Lite) to match a large teacher's soft logits and/or features.
- **Combine:** distill, then prune, then quantize; export with TFLite, CoreML or TensorRT.

Risks to watch:

- accuracy loss concentrated on **rare classes, small objects or hard slices** (check per-class and per-slice metrics, not just the average);
- **calibration** shifts;
- layers sensitive to quantization (first and last layers, depthwise convs, activations with outliers);
- BN folding errors;
- operator-support fallbacks that kill speed;
- mismatch between simulated and on-device numerics.

Always validate on the device with real data.

CNN internals and training details

DL03-120 How does backpropagation work through convolutional and pooling layers?

- **Convolution:** the forward pass is $y = x \star W$ (cross-correlation). Backward:
  - $\boldsymbol{\partial L/\partial W}$ = cross-correlation of the input x with the output gradient $\partial L/\partial y$ (summed over batch and positions, since weights are shared);
  - $\boldsymbol{\partial L/\partial x}$ = "full" convolution of $\partial L/\partial y$ with the kernel **rotated 180°**, which is a transposed convolution (with stride, the gradient is dilated by inserting zeros);
  - $\boldsymbol{\partial L/\partial b}$ = sum of $\partial L/\partial y$ over positions.
- **Max pooling:** the gradient flows only to the input position that was the max in each window (switch recorded in the forward pass); others get 0.
- **Average pooling:** the gradient is divided equally among the k^2 inputs of the window.
- **ReLU:** the gradient passes where the input > 0.

DL03-121 What are the advantages of deep CNNs over shallow ones, and what makes very deep CNNs hard to train?

Advantages of depth: hierarchical composition of features (edges → parts → objects), large receptive fields, more nonlinearities, so more expressive functions with fewer parameters than wide shallow networks (depth efficiency) and better generalization in practice.

Difficulties:

- vanishing and exploding gradients;
- the degradation problem (optimization gets harder);

- internal signal propagation and initialization sensitivity;
- more memory for activations and longer, sequential compute;
- overfitting with little data.

Solutions: residual connections, BatchNorm, careful initialization (He), good optimizers and learning-rate schedules, and regularization.

DL03-122 How do you choose the number and size of filters in a convolutional layer?

- **Size:** 3×3 is the default (stack them for larger receptive fields); 1×1 for channel mixing; a larger first-layer kernel (7×7 stride 2) for a cheap stem; dilation for context without extra cost; small kernels for fine detail; large kernels (7×7 depthwise, as in ConvNeXt) for wide context cheaply.
- **Number:** start small (32–64) and **double the channels each time resolution halves**, keeping compute per layer roughly balanced; widths that are multiples of 8 or 16 for hardware efficiency.
- **Tune** by validation accuracy versus FLOPs and latency; use known scaling rules (EfficientNet compound scaling, RegNet design-space findings) rather than ad-hoc choices.

DL03-123 What are the differences between 1D, 2D and 3D convolutions, and what data is each suited to?

The number refers to the dimensions the kernel slides over:

- **1D conv** slides over one axis (time or sequence): audio waveforms, sensor and accelerometer data, ECG, text (over tokens), time series.
- **2D conv** slides over height and width: images, spectrograms, 2D maps; channels are fully connected.
- **3D conv** slides over three axes (time × H × W, or depth × H × W): video clips (spatiotemporal), volumetric medical scans (CT and MRI), voxelized 3D data. Cost and memory grow by an extra K factor, so they are expensive; (2+1)D factorization or pseudo-3D are cheaper.

DL03-124 When would you choose a CNN over an RNN, and what are their typical use cases?

- **CNNs:** data with local, translation-invariant structure on a grid; fully parallel; fast. Use cases: images, video frames, spectrograms, local patterns in sequences (1D CNN or TCN for audio and sensors, short-text classification).
- **RNNs (LSTM, GRU):** sequential data where order and long-term state matter and inputs arrive step by step; streaming with constant memory. Use cases: language modeling (pre-transformer), speech recognition, time-series forecasting, handwriting.

Prefer CNNs (or TCNs) when parallel training and speed matter and dependencies are fairly local; prefer RNNs for online and streaming inference with unbounded context. Today transformers have replaced both for many long-range sequence tasks.

DL03-125 How is dropout applied in CNNs, and why is spatial dropout often preferred over standard dropout for conv features?

Standard dropout zeroes individual activations independently. In conv feature maps, neighboring pixels are **strongly correlated**, so a dropped pixel's information is recovered from its neighbors: dropout regularizes weakly and adds noise. **Spatial dropout (Dropout2d)** drops **entire channels** (feature maps), forcing the network not to rely on any single feature detector, which is a much stronger, more meaningful regularizer for convs. **DropBlock** drops contiguous spatial blocks for a similar reason. In modern CNNs, dropout is often used only before the final classifier; BN plus augmentation plus weight decay regularize the conv layers, and stochastic depth regularizes residual networks.

DL03-126 How do you normalize pixel values for a CNN, and why normalize with per-channel dataset mean and standard deviation?

Convert to float, scale to [0, 1] (divide by 255), then **standardize per channel**: $x' = (x - \mu_c) / \sigma_c$, e.g. ImageNet $\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$. **Why:**

- zero-centered, unit-variance inputs give well-conditioned gradients and faster, more stable optimization (matching weight-initialization assumptions);
- it equalizes channel scales so no channel dominates;
- with a pretrained model you **must use the same statistics** it was trained with, or the input distribution shifts and accuracy drops.

For other domains (medical, satellite), compute the statistics on your training set (or use domain-specific normalization such as CT windowing).

DL03-127 How do you handle images with different aspect ratios during preprocessing: resizing, cropping or padding?

- **Plain resize (stretch):** simple, keeps all content, but distorts shapes; acceptable if objects are large and distortion is consistent between training and test.
- **Resize short side, then center or random crop:** keeps aspect ratio but may cut off content at the edges; the standard for ImageNet classification.
- **Letterbox (resize long side, then pad):** keeps aspect ratio and all content; standard in detection (YOLO); wastes some compute on padding; box coordinates must be adjusted.
- **Alternatives:** aspect-ratio bucketing (batch similar ratios together), models that accept variable sizes (FCN, ViT with interpolated positions, NaViT), tiling for very large images.

Choose based on whether edge content matters and whether shape distortion changes the label; keep training and inference consistent.

DL03-128 What techniques can reduce the computation time of a CNN at training and inference time?

- **Architecture:** efficient backbones (MobileNet, EfficientNet, RegNet), depthwise separable or grouped convs, bottlenecks, early downsampling, fewer channels, smaller input resolution.
- **Numerics:** mixed precision (FP16 or BF16, tensor cores), INT8 quantization at inference.
- **Model compression:** structured pruning, distillation into a smaller student.
- **Systems:** channels-last memory format, cuDNN autotuning, operator fusion (conv + BN + ReLU), torch.compile or TensorRT or ONNX Runtime, larger batches for throughput, efficient data loading (GPU decode, prefetching, caching).
- **Algorithmic:** Winograd or FFT convolutions, early exit, cascades (cheap model first), progressive resizing during training, fewer epochs with better schedules.

DL03-129 How do attention modules such as squeeze-and-excitation or CBAM improve CNNs?

- **Squeeze-and-Excitation (SE):**
  1. **Squeeze** each channel to one value with global average pooling.
  2. **Excite** with a small MLP (reduce by $r = 16$, ReLU, expand, sigmoid), producing per-channel weights in (0, 1).
  3. **Rescale** the feature maps channel-wise.

  It models channel interdependencies and recalibrates which features matter for this input. About 1–2% ImageNet gain for under 1% extra compute.
- **CBAM:** channel attention (using both average- and max-pooled descriptors), followed by **spatial attention** (pool across channels, 7×7 conv, sigmoid map), so it emphasizes **what** and **where**.

Both add dynamic, input-dependent weighting and some global context to otherwise local convolutions.

DL03-130 What are the limitations of CNNs in capturing global context, and how do dilated convolutions, non-local blocks or attention address them?

Convolutions are **local**: global context only emerges after many layers, and the **effective receptive field** is much smaller than the theoretical one, so long-range relationships (two distant parts of an object, scene context) are hard to model. **Fixes:**

- **Dilated convolutions:** exponentially larger receptive field without downsampling (DeepLab).
- **Pyramid pooling and ASPP:** aggregate context at several scales.
- **Non-local blocks:** self-attention over all positions; each output is a weighted sum of features at every location ($y_i = \Sigma_j f(x_i, x_j)\, g(x_j)$).
- **Attention modules** (SE for global channel context, CBAM), transformers and hybrids (ViT, Swin, BoTNet), large-kernel convs (ConvNeXt 7×7, RepLKNet 31×31), global pooling branches.

DL03-131 What are capsule networks, and how do they differ from standard CNNs?

Capsule networks (Sabour, Frosst and Hinton, 2017) replace scalar neurons with **capsules**: vectors whose **length** encodes the probability that an entity exists and whose **orientation** encodes its pose (position, rotation, scale). Lower capsules predict higher capsules' poses through learned transformation matrices, and **dynamic routing by agreement** sends output to the parent capsules whose predictions agree. **Differences from CNNs:**

- they aim for **equivariance** (pose information preserved) instead of the invariance from max pooling, which discards spatial relationships;
- they model part–whole hierarchies explicitly;
- they generalize better to new viewpoints in small experiments.

Downsides: slow iterative routing, hard to scale, and no clear advantage on large benchmarks, so they remain mostly a research topic.

DL03-132 How can you visualize what a convolutional layer has learned, for example with filter visualization, activation maps or Grad-CAM?

- **Filter visualization:** plot first-layer kernels directly (edges, colors); deeper filters are not directly interpretable.
- **Activation maps:** show the feature maps for an input to see what each channel fires on; find the dataset images or patches that maximally activate a unit.
- **Activation maximization / feature visualization:** optimize an input image to maximize a neuron or channel, with regularizers (jitter, blur) for natural-looking patterns.
- **Saliency and attribution:** vanilla gradients, SmoothGrad, Integrated Gradients, occlusion sensitivity.
- **CAM and Grad-CAM:** weight the last conv feature maps by the global-average-pooled gradients of the class score, sum, apply ReLU and upsample, giving a class-specific heatmap of the image regions driving the prediction. Grad-CAM++ and Score-CAM are variants.

Used to debug shortcut learning, verify localization and explain models.

DL03-133 Why does batch normalization behave poorly with the small per-GPU batch sizes common in detection and segmentation, and what are the alternatives?

BN normalizes with **batch statistics**. Detection and segmentation use large images, so each GPU holds only 1–4 images and the per-batch mean and variance are **noisy, biased estimates**. That gives unstable training, poor running statistics and a train–inference mismatch. **Alternatives:**

- **SyncBN:** compute statistics across all GPUs.
- **Group Normalization:** normalizes over channel groups within each sample, independent of batch size (the Mask R-CNN and Detectron default).
- **Frozen BN:** use the pretrained statistics and affine parameters fixed, which is common when fine-tuning backbones.

- **LayerNorm** (transformers, ConvNeXt), Instance Norm (style tasks), batch renormalization, or weight standardization combined with GN.

DL03-134 Why does a CNN trained at one resolution often do better when tested at a slightly higher resolution, and how do you fix the train-test resolution discrepancy?

Training uses **RandomResizedCrop**, which zooms in, so objects look **larger** in training images than in center-cropped test images at the same resolution. The apparent object size differs between training and test (Touvron et al., FixRes). Testing at a somewhat higher resolution makes test objects match the training object scale, so accuracy improves. **Fixes:**

- **FixRes:** train at a lower resolution (cheaper), then **fine-tune the classifier (and BN statistics)** briefly at the test resolution;
- choose the test resolution about 1.15–1.3× the training resolution;
- progressive resizing; matching test-time crop ratios; multi-scale training.

Modern backbones and vision transformers

DL03-135 How have CNN architectures evolved from LeNet and AlexNet through VGG and ResNet to EfficientNet?

- **LeNet-5 (1998):** conv + pooling + FC for digits; established the core design.
- **AlexNet (2012):** deeper and wider, **ReLU**, **dropout**, data augmentation, GPU training, LRN; won ImageNet by a large margin and started the deep learning era.
- **ZFNet (2013):** tuned AlexNet using visualization.
- **VGG (2014):** uniform stacks of **3×3** convs, depth 16–19; simple but heavy (138M parameters).
- **GoogLeNet / Inception (2014):** multi-branch modules, **1×1 reductions**, global average pooling; efficient.
- **ResNet (2015): residual connections** plus BN; 50–152+ layers trainable; bottleneck blocks.
- **Later refinements:** DenseNet (dense connectivity), ResNeXt (grouped convs, "cardinality"), SENet (channel attention), MobileNet and ShuffleNet (efficient convs).
- **EfficientNet (2019):** NAS-designed MBConv blocks plus **compound scaling**; best accuracy per FLOP of its time.
- **Afterwards:** RegNet, NFNet, ConvNeXt (2022), and vision transformers.

The trend: deeper, then easier to optimize, then more efficient, then scaled systematically.

DL03-136 What design changes turned ResNet into ConvNeXt, and what does ConvNeXt show about CNNs versus ViTs?

ConvNeXt ("A ConvNet for the 2020s") modernized ResNet-50 step by step using Swin Transformer design choices:

- **training recipe:** AdamW, 300 epochs, Mixup, CutMix, RandAugment, stochastic depth, label smoothing;
- **macro design:** stage compute ratio 1:1:3:1, and a **"patchify" stem** (4×4 conv with stride 4);
- **depthwise convs** (like per-channel attention mixing), wider channels;
- **inverted bottleneck** (expand 4×);
- **large 7×7 depthwise kernels**, with the depthwise conv moved up;
- **fewer activations and norms**, **GELU** instead of ReLU, **LayerNorm** instead of BN;
- separate downsampling layers.

The result matched or beat Swin at the same FLOPs. **Lesson:** much of the ViT advantage came from training recipes and design choices, not attention itself; a pure CNN with modern choices is competitive and often faster and simpler to deploy.

DL03-137 How does the Swin Transformer use shifted windows to make self-attention efficient for high-resolution images?

Global self-attention over all patches is $O(N^2)$ in the number of tokens, which is infeasible for high-resolution images. **Swin:**

- computes self-attention **within local non-overlapping windows** (e.g. 7×7 patches), so cost is **linear in image size**: $O(N \cdot M^2)$ for window size M;
- in alternate layers, **shifts the window partition** by half a window (M/2), so tokens near window borders attend across the previous windows' boundaries, connecting windows and growing the receptive field (efficiently implemented with cyclic shift plus attention masks);
- uses **hierarchical patch merging** (2×2 neighboring patches concatenated and projected), halving resolution and doubling channels per stage, which produces multi-scale feature maps like a CNN (strides 4, 8, 16, 32), suitable for FPN-based detection and segmentation;
- adds a relative position bias.

DL03-138 How does patch size affect a Vision Transformer's accuracy, sequence length and compute?

For an H×W image and patch size P, the number of tokens is $N = HW / P^2$.

- **Smaller patches** (e.g. 8 instead of 16) give 4× more tokens: finer detail, typically **higher accuracy**, especially for dense tasks and small objects, but attention cost grows with N^2 (16× more attention FLOPs and memory) and MLP cost grows with N (4×).
- **Larger patches** (32): cheaper and faster, coarser, lower accuracy.

ViT naming (ViT-B/16, B/32) shows this trade-off; FlexiViT trains one model for multiple patch sizes.

DL03-139 How does self-attention cost scale with image resolution in a ViT, and why does that matter for dense prediction tasks?

Self-attention costs $O(N^2 \cdot D)$ FLOPs and $O(N^2)$ memory for the attention matrix, with $N = HW/P^2$. Doubling image resolution gives 4× the tokens and **16× the attention cost**. Dense prediction (detection, segmentation) needs **high-resolution inputs and fine feature maps** (e.g. 1024×1024 with stride 4–8), where N reaches tens of thousands of tokens, so global attention becomes prohibitively expensive in compute and memory. **Solutions:** windowed or local attention (Swin), deformable attention (attend to a few sampled points), hierarchical or pyramid ViTs with spatial-reduction attention (PVT), linear or efficient attention, FlashAttention to cut memory, and plain ViT backbones with windowed blocks plus a few global blocks (ViTDet).

DL03-140 How do you fine-tune a ViT at a higher resolution than it was pre-trained at, and what happens to its positional embeddings?

- Keep the **patch size** fixed, so a higher resolution gives **more tokens**.
- The learned 1D position embeddings were trained for a fixed grid (e.g. 14×14 for 224/16). Reshape them to 2D and **interpolate** (bicubic) to the new grid (e.g. 24×24 for 384), keeping the CLS embedding separate.
- Then **fine-tune** at the new resolution (usually with a lower learning rate) so the network adapts to the interpolated positions and the new object scale.
- Relative position bias tables (Swin) are interpolated similarly.

Higher-resolution fine-tuning typically improves accuracy (e.g. ViT at 384) at roughly 3× compute.

DL03-141 What is the role of the CLS token in a ViT, and how does it compare to global average pooling of patch tokens?

The CLS token is an extra **learnable embedding** prepended to the patch tokens. Through self-attention it aggregates information from all patches, and its final state is fed to the classification head (inherited from BERT). **Global average pooling** of the final patch tokens is an alternative: it is equally good or better with the right learning rate, it avoids a special token, and it is often preferred in newer models (DeiT III, MAE fine-tuning). CLS can learn an adaptive weighting of patches (attention maps from CLS give useful saliency, as in DINO); GAP treats all patches equally and is more translation-robust. Some models use both (e.g. distillation tokens).

DL03-142 When would you choose a CNN backbone over a ViT for a production vision system, and vice versa?

Choose a CNN when:

- data is limited and there is no strong pretrained ViT;
- deployment is on **edge or mobile** hardware with mature conv kernels and INT8 support;
- strict latency at high resolution (convs scale linearly with pixels);
- dense tasks needing multi-scale features cheaply;
- the team needs simplicity and robustness (BN folding, quantization).

Choose a ViT when:

- large-scale pretrained weights exist (CLIP, DINOv2, MAE) and transfer matters;
- abundant data or compute;
- you need global context or multimodal integration (VLMs);
- you want the best accuracy at server scale;
- the hardware has efficient attention (GPUs or TPUs with FlashAttention).

Hierarchical hybrids (Swin, ConvNeXt, EfficientViT) are a common compromise. Benchmark both on the target device and data.

DL03-143 What are hybrid CNN-transformer architectures, and what does each part contribute?

Hybrids combine convolutions and attention:

- a **conv stem or early stages** (e.g. ResNet stages, CoAtNet, the convolutional stem in ViTC) provide **locality, translation equivariance, efficient high-resolution processing** and faster, more stable training with less data;
- **transformer blocks in later stages** at low resolution provide **global context and content-dependent interactions** where N is small, so attention is affordable.

Examples: the original ViT-hybrid (ResNet features into ViT), BoTNet (self-attention replacing 3×3 convs in the last ResNet stage), CoAtNet, CvT (convolutional projections), MobileViT and EfficientViT (mobile), DETR (CNN backbone plus transformer). They often achieve better accuracy and efficiency trade-offs than pure CNNs or pure ViTs, especially at moderate data scale.

Self-supervised and vision-language representation learning

DL03-144 How does contrastive learning such as SimCLR learn image representations, and why does it need large batches or many negatives?

SimCLR:

1. Each image x gets two random augmentations (crop + resize, color jitter, blur), giving x_i and x_j, a **positive pair**.
2. An encoder (ResNet) plus a projection MLP maps them to z_i and z_j.

3. The **NT-Xent / InfoNCE loss** pulls positives together and pushes away all other 2(N − 1) samples in the batch (negatives): $\ell = -\log[\exp(\text{sim}(z_i, z_j)/\tau) / \Sigma_{k\neq i} \exp(\text{sim}(z_i, z_k)/\tau)]$, with cosine similarity and temperature τ.
4. After pretraining, drop the projection head and use the encoder features.

Why large batches or many negatives: the loss estimates a contrast against the data distribution. More negatives give a tighter mutual-information bound, harder negatives and a better-conditioned gradient, which prevents trivial solutions. SimCLR needed batches of 4096–8192. MoCo avoids the large batch with a **momentum encoder plus a queue** of past negatives (65k).

DL03-145 How do BYOL and DINO avoid representation collapse without negative pairs?

Without negatives, the trivial solution is collapse: every image maps to the same vector. They avoid it through **architectural asymmetry and target design**:

- **BYOL:** an online network (with an extra **predictor** MLP) predicts the output of a **target network** whose weights are an **exponential moving average** of the online weights, and the target receives **no gradient** (stop-gradient). The predictor plus slowly moving target creates dynamics that do not converge to collapse (SimSiam shows stop-gradient plus predictor alone suffices).
- **DINO:** self-distillation where a student matches the softmax output of an EMA teacher across multi-crop views (global and local crops). Collapse is prevented by **centering** the teacher outputs (subtract a running mean, which stops one dimension dominating) and **sharpening** them (low teacher temperature, which stops a uniform output). The two balance each other.

Other approaches: Barlow Twins and VICReg (decorrelation and variance terms).

DL03-146 Why does a masked autoencoder use a very high masking ratio such as 75%, and why is the encoder applied only to visible patches?

- **High masking (75%):** images are highly **spatially redundant**. With low masking, missing patches can be filled in by interpolating from neighbors, a trivial low-level task. Masking most of the image forces the model to reason about objects, shapes and context to reconstruct, which gives semantic representations. Contrast BERT, where ~15% suffices because language is information-dense.
- **Encoder on visible patches only:** with 75% masked, the encoder processes only 25% of the tokens, so **pretraining is ~3–4× faster** with lower memory, and there is **no train–test mismatch** from mask tokens (at fine-tuning the encoder sees real patches only). A **lightweight decoder** takes the encoded visible tokens plus learnable mask tokens with positions and reconstructs pixels (normalized per patch); the loss is computed on masked patches only. The decoder is discarded afterwards.

DL03-147 How do you evaluate a self-supervised visual representation with linear probing, k-NN and fine-tuning, and why can they disagree?

- **Linear probing:** freeze the encoder and train a linear classifier on its features. Measures **linear separability** of the frozen representation; cheap.
- **k-NN:** classify by nearest neighbors in feature space (cosine). No training; measures local neighborhood semantics; sensitive to feature scale and geometry.
- **Fine-tuning:** train all weights end to end on labeled data. Measures how good an **initialization** the model is; usually the highest accuracy.

They disagree because they test different properties. Contrastive and distillation methods (DINO, MoCo) produce linearly separable, k-NN-friendly features, so they are strong on linear probing and k-NN. MAE features are not linearly separable (strong on fine-tuning, weaker on linear probing) because reconstruction learns rich but non-linear, less invariant features. Report several protocols, plus low-shot and transfer to detection or segmentation.

DL03-148 How is CLIP trained, and what roles do the contrastive loss and the learned temperature play?

CLIP jointly trains an **image encoder** (ResNet or ViT) and a **text encoder** (transformer) on ~400M image–caption pairs.

1. Embed a batch of N images and N texts and project them to a shared space; L2-normalize.
2. Compute the N×N cosine-similarity matrix, scaled by a temperature: logits = $(I \cdot T^{\intercal}) \cdot \exp(t)$.
3. **Symmetric contrastive (InfoNCE) loss:** cross-entropy over rows (image to text) plus columns (text to image), where the diagonal entries are the correct pairs and the other $N - 1$ in each row and column are negatives. A very large batch (32k) provides many negatives.

Temperature (a learned logit scale, initialized at 1/0.07 and clipped at 100): it controls how peaked the softmax is. Too high a temperature gives soft distributions and weak separation; too low gives unstable, overconfident gradients. Learning it lets the model adjust the sharpness of the contrast as embeddings improve.

DL03-149 What are the known failure modes of CLIP, such as counting, spatial relations and typographic attacks?

- **Counting:** cannot reliably distinguish "three dogs" from "five dogs".
- **Spatial relations and compositionality:** "a horse on a man" versus "a man on a horse"; attribute binding ("red cube, blue sphere"); negation. It behaves like a bag of concepts.
- **Typographic attacks:** an apple with a handwritten label "iPod" is classified as an iPod, because CLIP reads text strongly.
- **Fine-grained and specialized domains:** car models, medical images, satellite imagery, digits (weak MNIST performance).
- **Abstract or systematic tasks:** distances, orientation, OCR of long text.
- **Biases** from web data (social stereotypes); sensitivity to **prompt wording**.
- **Weak localization** (global embedding only) and poor calibration on out-of-distribution data.

DL03-150 What is zero-shot image recognition, and how does it differ from few-shot learning in computer vision?

- **Zero-shot recognition:** classify categories with **no training images** of those classes, using side information that links classes: attribute vectors, word embeddings or, today, **text prompts with a vision-language model** (CLIP), which matches image embeddings to class-description embeddings.
- **Few-shot learning:** a **small number of labeled examples** per new class (e.g. 1–5, "N-way K-shot"). Methods: metric learning (prototypical or matching networks), meta-learning (MAML), fine-tuning or linear probing on strong pretrained features, adapter or prompt tuning of VLMs (CoOp, Tip-Adapter).
- **Generalized zero-shot:** test classes include both seen and unseen classes, which brings a bias toward seen classes.

DL03-151 What are the challenges of few-shot learning for image classification, and how do prototypical networks address them?

Challenges:

- overfitting with 1–5 examples per class;
- high variance of the estimates;
- intra-class variation not covered by the few shots;
- domain shift between base and novel classes;
- class imbalance between abundant base classes and rare novel ones.

Prototypical networks: learn an embedding f where classes form clusters. For each class, the **prototype** is the mean embedding of its support examples, $c_k = (1/K) \Sigma f(x_i)$. A query is classified by softmax over negative squared Euclidean distance to the prototypes. Training is **episodic** (sample N-way K-shot tasks from base classes) so it matches the test conditions. The approach is simple, has few parameters (no per-class training), and

averaging reduces noise. Modern practice shows that strong pretrained features plus a simple prototype or linear classifier is often as good as complex meta-learning.

Detection deep dive

DL03-152 What algorithms would you choose for real-time object detection, and why?

Choice depends on the hardware, accuracy and latency targets:

- **YOLO family** (YOLOv5/v8, YOLOX, YOLOv7/v9/v10/v11): best-supported real-time choice, 30–300 FPS depending on size (n/s/m/l/x), easy export to ONNX, TensorRT or CoreML.
- **RT-DETR:** a real-time transformer detector without NMS, competitive accuracy on GPUs.
- **SSD-MobileNet, EfficientDet-Lite, NanoDet, PicoDet:** for mobile CPUs and microcontrollers.
- **CenterNet or FCOS:** simple anchor-free options.

Reasons: single-stage, fully convolutional or simple heads, good accuracy per FLOP, mature deployment tooling, INT8 support. Select by benchmarking mAP against latency on the target device at the target input size, including NMS and preprocessing time.

DL03-153 How does YOLO perform detection in a single forward pass, and what changed across later YOLO versions?

YOLOv1: divide the image into an S×S grid (7×7). Each cell predicts B boxes (x, y, w, h, objectness) and class probabilities, all from **one forward pass** of one network, trained with a sum-squared regression loss. Very fast, but poor on small objects and crowds (each cell predicts few objects). **Later versions:**

- **v2:** BatchNorm, **anchor boxes** from k-means, higher resolution, multi-scale training, Darknet-19.
- **v3: multi-scale predictions** (FPN-like, 3 scales), Darknet-53, logistic class predictions.
- **v4/v5:** CSP backbones, PANet neck, Mosaic augmentation, CIoU loss, auto-anchors, a PyTorch ecosystem.
- **YOLOX / v6 / v8: anchor-free**, **decoupled** classification and regression heads, advanced label assignment (SimOTA, task-aligned assignment), distribution focal loss.
- **v7/v9:** re-parameterization, E-ELAN, programmable gradient information.
- **v10: NMS-free** training with consistent dual assignments.

Throughout: better accuracy–speed trade-offs and easier deployment.

DL03-154 How do Haar cascades and the Viola-Jones detector work, and why were they replaced by CNN detectors?

- **Haar features:** rectangular light–dark contrast patterns (edges, lines, center-surround), computed in O(1) using an **integral image**.
- **AdaBoost** selects a few discriminative features from ~160k candidates, combining weak classifiers (each a thresholded feature) into a strong classifier.
- **Attentional cascade:** a sequence of increasingly complex classifiers. Early stages quickly reject most background windows (with high recall); only promising windows reach later stages, enabling real-time sliding-window detection across scales (2001, face detection on 2000s hardware).

Why replaced: hand-crafted features are limited; poor robustness to pose, lighting, occlusion and in-plane rotation; one detector per object class and viewpoint; many false positives. CNN detectors learn features end to end, handle many classes and variations, and are far more accurate while still running in real time on GPUs and NPUs.

DL03-155 How do anchor-free detectors such as FCOS or CenterNet work, and what problems with anchors do they avoid?

- **FCOS:** each feature-map location **inside** a ground-truth box predicts the class, the **distances to the four box edges (l, t, r, b)** and a **centerness** score (which down-weights low-quality predictions far from the object center). An FPN assigns objects to pyramid levels by size range.
- **CenterNet ("objects as points"):** predicts a **heatmap of object centers** per class (Gaussian-splatted targets, focal loss) plus box size and center offset at each peak. Peaks are extracted with 3×3 max pooling, so no NMS is needed in the simple setting.

Problems with anchors they avoid:

- many anchor hyperparameters (scales, ratios, IoU thresholds) that must be tuned per dataset;
- severe positive–negative imbalance from huge numbers of anchors;
- extra compute for IoU matching;
- poor generalization to unusual shapes;
- ambiguous assignment.

Anchor-free detectors are simpler and generalize better, and they now dominate (YOLOX and v8, FCOS-based designs).

DL03-156 How does DETR use Hungarian matching to remove the need for NMS, and why does it converge slowly?

- **Hungarian matching:** DETR predicts a fixed set of N predictions. During training it computes the optimal **one-to-one bipartite matching** between predictions and ground-truth objects (padded with "no object") by minimizing a matching cost: −class probability + $\lambda_1 \cdot$L1(box) + $\lambda_2 \cdot (1 - \text{GIoU})$. Each object is matched to exactly **one** prediction; unmatched predictions are trained to predict "no object". Duplicates are penalized, and decoder self-attention lets queries communicate to avoid them, so **NMS is unnecessary**.
- **Slow convergence (~500 epochs on COCO):**
  - initially uniform cross-attention over all image positions must learn to focus on sparse object regions, which is hard;
  - the Hungarian assignment is **unstable** early on (the same query matches different objects from epoch to epoch);
  - object queries carry no positional prior.
- **Fixes:** Deformable DETR (attend to a few sampled points, multi-scale), conditional or DAB-DETR (anchor-like positional queries), DN-DETR and DINO (denoising training with noised ground-truth boxes), and one-to-many auxiliary assignments (H-DETR, Co-DETR). These cut training to around 12–50 epochs.

DL03-157 What is soft-NMS, and when does it beat standard non-maximum suppression?

Standard NMS **deletes** any box whose IoU with a higher-scoring box exceeds the threshold. In crowds or with overlapping objects, a true detection of a second object is suppressed, which lowers recall. **Soft-NMS** instead **decays the score** of overlapping boxes according to their overlap:

- linear: $s_i \leftarrow s_i \cdot (1 - \text{IoU})$ when $\text{IoU} > N_t$;
- Gaussian: $s_i \leftarrow s_i \cdot \exp(-\text{IoU}^2 / \sigma)$.

Then it continues by score. Heavily overlapping duplicates get low scores, while moderately overlapping true objects survive with reduced scores, which a later threshold or the AP computation can still use. It beats standard NMS when objects **overlap significantly** (pedestrians, crowds, cars in traffic, densely packed retail items) and under mAP averaged over strict IoU thresholds. It is a drop-in change with no retraining; it may slightly increase false positives at low thresholds.

DL03-158 How is COCO mAP computed across IoU thresholds and object sizes, and how is it different from PASCAL VOC mAP?

COCO:

- AP is computed per class at **10 IoU thresholds, 0.50:0.05:0.95**, each using **101-point interpolated precision** (recall 0, 0.01, …, 1), then averaged over thresholds and over the 80 classes. The resulting "AP" is mAP@[.5:.95].
- Also reports **AP50** and **AP75**, and **AP_S / AP_M / AP_L** by object area ($< 32^2$, 32^2–96^2, $> 96^2$ pixels), plus AR (average recall) with a maximum of 1, 10 or 100 detections per image.
- Crowd regions are marked "iscrowd" and ignored.

PASCAL VOC: a single IoU threshold of **0.5** (AP50); VOC2007 used **11-point** interpolation, VOC2010+ the area under the full monotone PR curve; 20 classes; "difficult" objects are ignored. COCO's averaging over strict IoUs rewards **precise localization**, while VOC mainly rewards finding objects.

DL03-159 How does label assignment, deciding which predictions are positives during training, affect detector quality?

Label assignment decides which anchors, points or queries are trained as positives (with which ground-truth box) and which as negatives. It shapes **what the network learns**:

- **Too few positives:** weak supervision and slow learning. **Too many low-quality positives:** noisy box regression, duplicates, poor localization.
- Fixed IoU thresholds are brittle across object sizes and shapes (small objects get few positives).
- A misalignment between classification score and localization quality means NMS keeps the wrong box.

Better strategies:

- **ATSS:** adaptive IoU threshold per object from the statistics of candidate IoUs.
- **OTA / SimOTA:** treat assignment as optimal transport using classification plus regression cost.
- **TAL (task-aligned learning, TOOD and YOLOv8):** the alignment metric $s^{\alpha} \cdot u^{\beta}$ uses both the classification score and the IoU.
- **Hungarian one-to-one assignment** (DETR), which removes NMS.

Changing only the assignment often yields 1–3 AP or more with no inference cost.

DL03-160 How do detection models deal with occlusion and partially visible objects?

- **Training data:** include occluded examples labeled with full ("amodal") or visible boxes consistently; occlusion augmentation (Cutout, random erasing, copy-paste of occluders).
- **Context and part-based reasoning:** large receptive fields and attention let the model infer an object from visible parts and context; part-based models and multiple heads (full body plus visible region, as in CrowdDet).
- **Post-processing:** soft-NMS, adaptive NMS, or set-based (NMS-free) prediction, so overlapping objects are not suppressed; repulsion loss to keep predicted boxes apart in crowds.
- **Temporal information** in video (tracking keeps identity through brief occlusions); multi-view or sensor fusion.
- **Amodal segmentation** predicts the hidden extent.
- **Evaluate** on occlusion-specific subsets (CrowdHuman, CityPersons heavy-occlusion split).

DL03-161 How do open-vocabulary detectors such as OWL-ViT or Grounding DINO detect classes never seen in the detection training set?

They replace the fixed classification layer with **text embeddings**:

- **OWL-ViT:** start from CLIP; remove token pooling so each ViT output token predicts a box, then project the token embeddings into CLIP's joint space. Class "weights" are text embeddings of arbitrary queries ("a photo of a giraffe"), or image embeddings for image-conditioned queries. Fine-tuned on detection data while keeping image–text alignment.

- **Grounding DINO:** a DINO-DETR detector with **deep vision–language fusion** (feature enhancer, language-guided query selection, cross-modality decoder), trained on detection plus **phrase-grounding** plus caption data, so it localizes any phrase in the prompt.

Generalization to unseen classes comes from **large-scale image–text pretraining** (concepts learned from captions) plus region–word alignment. Limitations: lower accuracy than closed-set detectors on known classes; sensitivity to prompt wording; confusion between similar concepts; weaker results on rare or specialized categories.

DL03-162 How would you handle a long-tailed detection dataset where a few classes have only a handful of boxes?

- **Resampling:** repeat-factor sampling (LVIS) oversamples images containing rare classes; class-aware sampling.
- **Loss re-weighting:** equalization loss (ignore suppressing gradients from frequent classes on rare ones), seesaw loss, focal loss, federated loss; logit adjustment or balanced softmax.
- **Decoupled training:** train the representation normally, then re-balance or retrain the classifier head with balanced sampling (cRT, BAGS).
- **Data:** copy-paste augmentation of rare instances, synthetic rendering, mining more examples (active learning), merging fine classes.
- **Few-shot detection techniques:** fine-tune only the last layers on balanced data (TFA), cosine classifiers.
- **Open-vocabulary or VLM-based detectors** that leverage text knowledge for rare categories.
- **Evaluate** with per-class AP and AP_rare / AP_c / AP_f.

DL03-163 How does test-time augmentation work for detection, and how do you merge the resulting boxes?

TTA for detection:

1. Run the detector on several transformed versions of the image: horizontal flip, multiple scales (e.g. 0.8×, 1×, 1.2×), sometimes small crops or color variants.
2. **Invert the transform on the predicted boxes**: unflip x coordinates, rescale to the original size.
3. **Merge** the pooled detections:
   - **NMS** or soft-NMS over the union (simple; keeps the single best box);
   - **Weighted Boxes Fusion (WBF):** cluster boxes by IoU and average their coordinates weighted by confidence, adjusting the score by how many models or augmentations agreed. Usually the most accurate, giving better localization (higher AP75).

It costs k× the inference time; it is useful for offline or competition settings, rarely for real time.

Segmentation deep dive

DL03-164 What is panoptic segmentation, and how is panoptic quality (PQ) computed?

Panoptic segmentation gives every pixel a semantic class **and**, for countable "things", an instance ID. "Stuff" (sky, road) has only a class. Segments must be non-overlapping. **Panoptic quality:** predicted and ground-truth segments of the same class are **matched if IoU > 0.5** (this makes the matching unique). Then:

- PQ = Σ_(TP) IoU(p, g) / (|TP| + ½|FP| + ½|FN|);
- PQ = **SQ × RQ**, where segmentation quality SQ = mean IoU of the matched pairs, and recognition quality RQ = |TP| / (|TP| + ½|FP| + ½|FN|), an F1-like detection score.

It is computed per class and averaged (also reported separately as PQ_things and PQ_stuff).

DL03-165 How do DeepLab's atrous convolutions and ASPP capture multi-scale context?

- **Atrous (dilated) convolutions** let DeepLab remove the last downsampling strides (output stride 16 or 8 instead of 32) while keeping large receptive fields and reusing pretrained weights, so features stay dense and high resolution without extra parameters.
- **ASPP (Atrous Spatial Pyramid Pooling):** parallel branches on the same feature map:
  - a 1×1 conv;
  - three 3×3 atrous convs with different rates (6, 12, 18 at output stride 16);
  - **image-level pooling** (global average pool, 1×1 conv, upsample) for global context.

  They are concatenated and fused with a 1×1 conv. Each branch sees a different effective field of view, capturing objects and context at multiple scales.
- **DeepLabv3+** adds a simple decoder that fuses low-level features for sharper boundaries.

DL03-166 How does Mask2Former unify semantic, instance and panoptic segmentation with mask classification?

Mask2Former treats **all segmentation tasks as mask classification**: predict a set of N binary masks, each with one class label (including "no object").

- **Backbone plus pixel decoder** (multi-scale deformable attention) produce per-pixel embeddings at several resolutions.
- **Transformer decoder:** N learnable queries; each query outputs a class and a mask embedding, and the mask = sigmoid(query embedding · per-pixel embeddings).
- **Masked attention:** each query's cross-attention is restricted to the foreground of its own predicted mask from the previous layer, which gives faster convergence and better small objects.
- **Training:** Hungarian matching between predicted and ground-truth masks; losses are cross-entropy for the class plus Dice and binary cross-entropy for the mask (computed on sampled points for efficiency).

Unification: semantic segmentation merges masks by class; instance segmentation takes each mask as an instance; panoptic segmentation combines both. One architecture achieves state of the art on all three.

DL03-167 What does the Segment Anything Model (SAM) do, and how does promptable segmentation differ from class-specific segmentation?

SAM (Segment Anything) is a **promptable segmentation foundation model**: given an image and a **prompt** (points, a box, a rough mask; text was explored), it outputs valid object masks.

- **Architecture:** a heavy **ViT image encoder** (MAE-pretrained, run once per image), a lightweight **prompt encoder** and a fast **mask decoder** (two-way attention, about 50 ms), which outputs **3 masks** for ambiguous prompts (whole, part, sub-part) with predicted IoU scores.
- **Data:** SA-1B, 11M images and 1.1B masks, built with a model-in-the-loop data engine.

Difference from class-specific segmentation: SAM segments "whatever the prompt points to", with **no class labels** and class-agnostic, zero-shot generalization to new objects and domains. Class-specific models (DeepLab, Mask R-CNN) output fixed categories and need labeled training data for each class. For semantic labels SAM must be combined with a detector or classifier (e.g. Grounding DINO + SAM). SAM 2 extends this to video with memory.

DL03-168 How can depth information from RGB-D sensors be used to improve semantic segmentation?

Depth gives **geometry** that RGB lacks: object boundaries where depth jumps, surface normals, height above the floor, 3D shape, all invariant to lighting and texture.

- **Input fusion (early):** stack depth as a 4th channel, or encode it as HHA (horizontal disparity, height, angle with gravity) to be more CNN-friendly.
- **Two-stream networks:** separate RGB and depth encoders fused at multiple levels (FuseNet, RedNet, ACNet), with attention-based fusion.

- **Depth-aware operations:** depth-aware convolution and pooling (weight neighbors by depth similarity), 3D point-cloud or voxel processing.
- **Multi-task learning:** predict depth and semantics jointly.

It helps separate objects with similar color, recognize floors and walls, and work in poor lighting. Challenges: noisy or missing depth (reflective or transparent surfaces), RGB–depth alignment, extra sensor cost.

DL03-169 Why do segmentation models often produce poor boundaries, and how would you improve boundary quality?

Causes:

- downsampling (stride 16–32) and bilinear upsampling blur fine detail;
- the loss is dominated by interior pixels;
- annotation noise at edges;
- class imbalance at thin structures.

Fixes:

- **Architecture:** keep high resolution (HRNet), skip connections and decoders (U-Net, DeepLabv3+), multi-scale fusion.
- **Refinement:** PointRend (predict at adaptively selected uncertain points with fine-grained features), CRF or dense-CRF post-processing, guided filters, SegFix-style boundary offset correction, cascade refinement.
- **Losses:** boundary-aware losses (boundary loss, active contour, Hausdorff), Lovász, weighted cross-entropy near edges, auxiliary edge-prediction tasks.
- **Data:** higher-quality boundary annotations, higher input resolution.
- **Evaluate** with boundary F1 or boundary IoU to track progress.

DL03-170 Pixel-level labels are expensive. How would you train a segmentation model with weak labels such as boxes, points or image tags?

Weakly supervised segmentation:

- **Image-level tags:** train a classifier, extract **class activation maps** (CAM, Grad-CAM) as seed regions, expand and refine them (affinity propagation such as AffinityNet or IRNet, CRFs, adversarial erasing to cover whole objects), use them as **pseudo-masks** and retrain a segmentation network; iterate.
- **Boxes:** generate masks inside boxes (GrabCut, or SAM prompted with the box); use the box-tightness prior (BoxInst: projection loss on box edges plus pairwise color-affinity loss); multiple-instance learning.
- **Points or scribbles:** partial cross-entropy on labeled pixels plus regularization that propagates labels to similar neighboring pixels (CRF loss, normalized cut, random walk); SAM with point prompts.
- **Combine with:** self-training and pseudo-label filtering by confidence, semi-supervised learning with a few full masks, and foundation models (SAM, DINOv2 features) to generate high-quality pseudo-labels.

Classical vision and geometry

DL03-171 How does the Histogram of Oriented Gradients (HOG) descriptor work?

HOG (Dalal and Triggs, 2005):

1. Optional gamma or color normalization.
2. Compute gradients (simple [−1, 0, 1] filters), giving magnitude and orientation per pixel.
3. Divide the image into **cells** (e.g. 8×8 pixels). For each cell, build a **histogram of gradient orientations** (9 bins over 0–180°, unsigned), with votes weighted by gradient magnitude and interpolated between bins.
4. Group cells into overlapping **blocks** (2×2 cells) and **normalize** each block (L2-Hys) for illumination and contrast invariance.
5. Concatenate all block histograms into one feature vector (3,780-d for a 64×128 pedestrian window).
6. Classify with a linear SVM in a sliding-window detector.

It captures **local shape and edge structure**, is robust to small deformations and lighting changes, and is not rotation-invariant.

DL03-172 Compare SIFT, SURF, ORB and HOG as hand-crafted feature extractors in speed, invariance and typical use.

	Speed	Invariance	Descriptor	Typical use
SIFT	Slow	Scale, rotation, illumination; partly affine	128-d float (L2)	Precise matching, stitching, SfM, 3D reconstruction
SURF	~3× faster than SIFT	Scale, rotation (Hessian keypoints, box filters, integral images)	64-d float	Faster SIFT-like matching (patented)
ORB	Very fast (real time)	Rotation (oriented FAST), scale via pyramid; less robust	256-bit binary (Hamming)	SLAM (ORB-SLAM), mobile AR, tracking
HOG	Moderate	Illumination (block norm); not rotation or scale invariant	Dense grid histograms	Object detection in a sliding window (pedestrians)

SIFT, SURF and ORB are **sparse keypoint** detectors and descriptors for matching. HOG is a **dense window** descriptor for classification and detection. Learned features (SuperPoint plus SuperGlue or LightGlue) now outperform them for matching.

DL03-173 How does the Hough transform detect lines and circles?

The Hough transform detects shapes by **voting in parameter space**:

- **Lines:** each line is parameterized as $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ (avoids infinite slopes). For each edge pixel (from Canny), vote for all (ρ, θ) bins consistent with it, which traces a sinusoid in the accumulator. Lines appear as **peaks** where many sinusoids intersect; threshold and apply non-maximum suppression. The probabilistic Hough transform samples points and returns segments.
- **Circles:** parameters (a, b, r). Each edge pixel votes for possible centers at radius r, a 3D accumulator, so the Hough gradient method uses edge direction to vote only along the gradient line, which reduces cost.

Pros: robust to noise, gaps and occlusion. **Cons:** memory and time grow with the number of parameters; bin resolution and threshold tuning are needed.

DL03-174 What are morphological operations such as erosion, dilation, opening and closing, and when are they used?

Morphological operations are nonlinear operations on binary (or grayscale) images using a **structuring element** (kernel shape):

- **Erosion:** a pixel stays 1 only if the whole kernel fits inside the foreground. Shrinks objects, removes small noise and thin protrusions.
- **Dilation:** a pixel becomes 1 if the kernel touches any foreground. Grows objects, fills small holes and gaps.
- **Opening** = erosion then dilation. Removes small specks and noise while roughly preserving shape.
- **Closing** = dilation then erosion. Fills small holes and joins nearby fragments.
- **Others:** morphological gradient (dilation − erosion) gives outlines; top-hat and black-hat extract small bright or dark details; skeletonization.

Uses: cleaning segmentation masks and thresholded images, OCR and document preprocessing, separating touching objects (with distance transform and watershed), counting cells, defect inspection.

DL03-175 What is histogram equalization, and when would you use CLAHE instead?

- **Histogram equalization** remaps intensities using the **cumulative distribution** (CDF) of the image histogram so output intensities are spread roughly uniformly, which increases global contrast. Problems: it amplifies noise, over-enhances, and a single global mapping fails when parts of the image differ (a dark region next to a bright one).
- **CLAHE** (Contrast-Limited Adaptive Histogram Equalization) equalizes **locally** on tiles (e.g. 8×8 grid), **clips** each histogram at a limit (redistributing the excess) to prevent noise amplification, and **bilinearly interpolates** between tiles to avoid block artifacts.

Use CLAHE for images with **non-uniform lighting or local low contrast**: medical X-ray and retinal images, underwater, night and foggy scenes, OCR of unevenly lit documents. Apply it to the luminance channel (LAB L) for color images.

DL03-176 What are image pyramids, and how are they used in multi-scale detection and blending?

An image pyramid is a set of progressively **downsampled** copies of an image:

- **Gaussian pyramid:** blur, then subsample by 2 at each level.
- **Laplacian pyramid:** the difference between a Gaussian level and the upsampled next level, storing band-pass detail; the image can be reconstructed exactly.

Uses:

- **Multi-scale detection:** run a fixed-size detector (sliding window HOG or Viola–Jones) at each pyramid level to find objects of different sizes; SIFT's DoG scale space; coarse-to-fine optical flow and registration (handle large motion at low resolution, refine at high). CNN feature pyramids (FPN) are the learned analogue.
- **Blending:** Laplacian pyramid blending of two images with a mask pyramid blends low frequencies over wide regions and high frequencies over narrow ones, giving seamless panoramas and composites without visible seams.
- Also compression and image editing at multiple scales.

DL03-177 What is a homography, and how do you estimate it robustly with RANSAC for panorama stitching?

A **homography** H is a 3×3 projective transformation (8 degrees of freedom, defined up to scale) mapping points between two images of a **planar scene**, or of any scene when the camera **only rotates** about its center: $x' \sim H x$ in homogeneous coordinates.

Robust estimation for stitching:

1. Detect and describe keypoints (SIFT, ORB) in both images; match them with the ratio test.
2. **RANSAC:** repeatedly pick **4 random correspondences** (the minimum, no three collinear), compute H by the **DLT** (direct linear transform, solved by SVD), count inliers whose reprojection error $\|x' - Hx\|$ is below τ pixels; keep the H with the most inliers.
3. Refine H on all inliers (least squares or Levenberg–Marquardt, with normalized coordinates).
4. Warp one image into the other's frame and blend (feathering or multi-band blending); for many images, run bundle adjustment.

DL03-178 What are a camera's intrinsic and extrinsic parameters, and how is a camera calibrated?

Projection model: $x \sim \mathbf{K}\,[\mathbf{R} \mid \mathbf{t}]\,X$, mapping world point X to pixel x.

- **Intrinsics K** (camera-internal): focal lengths f_x and f_y (in pixels), principal point (c_x, c_y), skew s (usually 0), plus **lens distortion** coefficients (radial k_1, k_2, k_3 and tangential p_1, p_2). K = [[f_x, s, c_x], [0, f_y, c_y], [0, 0, 1]].
- **Extrinsics [R | t]:** the camera's rotation and translation relative to the world (pose), 6 degrees of freedom.

Calibration (Zhang's method):

1. Capture many images of a known pattern (checkerboard or ChArUco) in different poses.

2. Detect corners with subpixel accuracy.
3. Each view gives a homography between the pattern plane and the image; the closed-form solution gives K, and then per-view extrinsics.
4. Refine all parameters (including distortion) by **nonlinear least squares** minimizing reprojection error (bundle adjustment).

Check the RMS reprojection error (< 0.5 px is good). Stereo or multi-camera calibration also estimates the relative extrinsics between cameras.

DL03-179 What is epipolar geometry, and how are the fundamental and essential matrices used in stereo vision?

Epipolar geometry describes the relationship between two views of a 3D scene. A point x in image 1 must lie, in image 2, on its **epipolar line** $l' = F x$, because the 3D point lies on the ray through x. All epipolar lines pass through the **epipoles** (the image of the other camera center).

- **Fundamental matrix F** (3×3, rank 2, 7 degrees of freedom) works in **pixel** coordinates: $x'^{\top} F x = 0$. It is estimated from ≥ 7–8 correspondences (eight-point algorithm with normalization, inside RANSAC) **without knowing the intrinsics**.
- **Essential matrix E** works in **normalized** (calibrated) coordinates: $E = K'^{\top} F K = [t]_{\times} R$ (5 degrees of freedom). It can be decomposed into the relative rotation R and translation direction t (four solutions; pick the one with points in front of both cameras).

Uses in stereo: recover the relative camera pose; **rectify** images so epipolar lines become horizontal rows, reducing correspondence search to 1D; triangulate 3D points; reject bad matches.

DL03-180 How does neural style transfer use content and style losses, and why does the Gram matrix capture style?

Neural style transfer (Gatys et al.) optimizes an image x to match the **content** of image c and the **style** of image s, using a fixed pretrained VGG:

- **Content loss:** squared error between feature maps of x and c at a deep layer (e.g. conv4_2), which preserves object layout and structure.
- **Style loss:** for several layers, compute **Gram matrices** $G^l_{ij} = \Sigma_k F^l_{ik} F^l_{jk}$ (inner products between channel feature maps over spatial positions), and minimize the squared differences between the Gram matrices of x and s.
- **Total:** $\alpha \cdot L_{content} + \beta \cdot L_{style}$ (+ total-variation smoothness), optimized over the pixels of x with L-BFGS or Adam.

Why the Gram matrix captures style: it records **which features co-occur** (correlations between channels: textures, colors, brush-stroke patterns) while **summing over positions**, so it discards **where** things are. That spatially invariant summary of texture statistics is "style". Fast feed-forward variants (Johnson et al.) train a network to do this in one pass; AdaIN matches channel means and variances for arbitrary styles.

Video, tracking, pose and 3D

DL03-181 How can CNNs be used for human activity recognition in video, and how do two-stream networks, 3D convolutions and video transformers compare?

- **Frame-level 2D CNNs:** classify frames and pool over time (average, TSN segments). Cheap but weak at motion.
- **Two-stream networks:** a spatial stream on RGB frames (appearance) plus a temporal stream on stacked **optical flow** (motion), fused late. Strong motion modeling, but optical flow is expensive to compute.
- **3D CNNs** (C3D, I3D with ImageNet weights inflated to 3D, SlowFast, X3D): spatiotemporal kernels learn appearance and motion jointly, end to end. Heavy compute and memory, limited temporal range (16–64 frames).

- **Video transformers** (TimeSformer with divided space-time attention, ViViT, Video Swin, VideoMAE): attention models long-range temporal dependencies and scales well with pretraining (VideoMAE, CLIP-initialized). Quadratic token cost requires factorized or windowed attention.

Trade-off: two-stream gives good motion cues at the cost of computing flow; 3D CNNs are a solid, efficient default for short clips; transformers give the best accuracy with large-scale pretraining and longer context.

DL03-182 How can CNNs be combined with RNNs, as in CNN-LSTM models, to model video or image sequences?

A **CNN extracts per-frame features** (a pretrained backbone, frame to vector), and an **RNN (LSTM or GRU) models the temporal sequence** of those vectors. The final hidden state, or all hidden states, feed a classifier or a sequence decoder (LRCN, Donahue et al.). **Uses:** action recognition, video captioning, gesture recognition, sign language, frame-level event detection, image-sequence prediction (ConvLSTM keeps spatial structure in the recurrent state, for weather nowcasting and video prediction). **Pros:** flexible sequence length, reuses image pretraining, cheaper than 3D CNNs. **Cons:** late temporal fusion misses fine motion; sequential RNNs are slow and limited at long range. 3D CNNs and transformers have largely replaced them.

DL03-183 How do (2+1)D convolutions factor a 3D convolution, and why can they train better?

A (2+1)D convolution (R(2+1)D, Tran et al.) replaces a t×d×d 3D kernel with:

1. a **2D spatial conv** (1×d×d), then
2. a **1D temporal conv** (t×1×1),

with a nonlinearity in between and the number of intermediate channels chosen to keep the parameter count about equal to the 3D conv. **Why it trains better:**

- it **doubles the number of nonlinearities**, so the network is more expressive at the same cost;
- **spatial and temporal factors are decoupled**, which makes optimization easier (lower training error, as reported);
- the spatial part can be initialized from 2D ImageNet models.

It gave better accuracy than full 3D convs on Kinetics and Sports-1M. Related designs: P3D, S3D, and factorized attention in TimeSformer.

DL03-184 How are multi-object tracking systems evaluated with metrics such as MOTA, IDF1 and HOTA?

- **MOTA (Multiple Object Tracking Accuracy)** = 1 − (FN + FP + IDSW) / GT, summed over frames, where IDSW counts identity switches. It is dominated by **detection** quality (FP and FN), barely reflects association, and can be negative.
- **IDF1** = 2·IDTP / (2·IDTP + IDFP + IDFN). It measures how consistently the **same identity** is kept over time (global one-to-one matching of trajectories), so it is association-focused.
- **HOTA (Higher Order Tracking Accuracy)** = √(DetA · AssA), averaged over localization thresholds α. It explicitly balances detection accuracy, association accuracy and localization, and decomposes into interpretable sub-metrics. It is now the standard (MOTChallenge, KITTI).

Also: MOTP (localization precision), ID switches, fragmentations, MT/ML (mostly tracked or mostly lost trajectories).

DL03-185 What is pose estimation, and how do top-down and bottom-up approaches differ?

Pose estimation predicts the locations of **body keypoints** (joints: shoulders, elbows, knees) in 2D or 3D, often connected into a skeleton. It also covers hand, face and animal poses and **6-DoF object pose**.

- **Top-down:** detect each person first, then estimate keypoints in each cropped box with a single-person model (heatmap regression: SimpleBaseline, HRNet, ViTPose). More accurate, especially for small people; cost grows with the number of people; fails when the detector misses or boxes overlap.

- **Bottom-up:** detect **all keypoints in the image at once**, then **group** them into individuals (OpenPose part affinity fields, associative embedding, HigherHRNet). Runtime is roughly constant with crowd size and robust in crowds; grouping is harder and less accurate for small or overlapping people.
- **One-stage or direct regression** (YOLO-Pose, DEKR, RTMPose variants) blur the line between the two.

Applications: sports and fitness analytics, AR, motion capture and animation, action recognition, human–robot interaction, ergonomics.

DL03-186 How does stereo depth estimation work, and why is monocular depth estimation fundamentally ambiguous?

- **Stereo:** two calibrated, rectified cameras separated by baseline B. For each pixel, find the corresponding pixel along the same row in the other image; the horizontal shift is the **disparity d**. Depth **Z = f·B / d** (f is the focal length in pixels). Matching uses block matching or SGM (semi-global matching) with cost aggregation, or deep stereo networks (cost volumes such as PSMNet, RAFT-Stereo). Accuracy falls with distance (error $\propto Z^2$), and textureless, reflective or occluded regions are hard.
- **Monocular ambiguity:** a single image is a **projection that discards depth**. Infinitely many 3D scenes (a small near object or a large far one) produce the same image, so absolute **scale is unobservable**. Monocular networks (MiDaS, Depth Anything, ZoeDepth) rely on **learned priors**: object sizes, perspective, texture gradients, occlusion, defocus, shading. They give good relative depth but metric depth only with known camera intrinsics, scene priors or extra cues, and they can fail on unusual scenes or optical illusions.

DL03-187 How does PointNet process unordered point clouds, and how does it achieve permutation invariance?

PointNet takes N points (x, y, z, plus features) directly, without voxelizing:

1. A **shared MLP** is applied independently to each point (per-point features, the same weights for every point).
2. A **symmetric aggregation function**, **max pooling** across points, produces a global feature vector. Because max is invariant to input order, the network is **permutation invariant**: any ordering of the same points gives the same output.
3. A classification MLP runs on the global feature; for segmentation, the global feature is concatenated back to each point feature.

Optional **T-Nets** predict alignment transforms (input and feature) for rough rotation invariance. Theory: max pooling plus MLPs can approximate any continuous symmetric set function. **Limitation:** no local neighborhood structure. **PointNet++** adds hierarchical local grouping (farthest point sampling plus ball query plus mini-PointNets) to capture local geometry at multiple scales.

DL03-188 What is a bird's-eye-view (BEV) representation, and why do autonomous driving stacks use it?

A **BEV** representation projects the 3D world onto a **top-down 2D grid** around the ego vehicle (e.g. 200×200 cells covering ±50 m), with features or semantic layers per cell. **Why driving stacks use it:**

- it is the **natural frame for planning and control**, since driving happens on the ground plane;
- it is a **unified space for fusing** multiple cameras, lidar and radar, plus HD maps and time (ego-motion alignment is simple);
- objects keep their **true metric size and position** without perspective distortion or scale changes with distance, and occlusion handling is cleaner;
- it suits many tasks (3D detection, lane and map segmentation, occupancy, motion forecasting) from one shared feature map.

Challenges: cameras need a learned view transform (depth estimation or attention, as in LSS and BEVFormer), resolution is limited at long range, and heights are lost (3D occupancy grids extend it).

DL03-189 How do NeRF and 3D Gaussian splatting represent a scene, and what are their tradeoffs in quality and rendering speed?

- **NeRF:** an MLP maps a 3D position and viewing direction to **density and view-dependent color**. Images are rendered by **volume rendering**: sample points along each camera ray and alpha-composite their colors. It is trained per scene from posed images with a photometric loss (positional encoding for high-frequency detail). **Quality:** excellent, continuous, compact, handles view-dependent effects. **Speed:** slow training (hours) and rendering (seconds per frame), because hundreds of MLP evaluations are needed per ray. Speedups: Instant-NGP hash grids, Plenoxels, distillation.
- **3D Gaussian Splatting:** represents the scene as **millions of explicit 3D Gaussians** (position, covariance or anisotropic shape, opacity, spherical-harmonic color), initialized from SfM points and optimized with **differentiable rasterization**, sorting and alpha-blending projected 2D Gaussians. **Speed:** real-time rendering (100+ FPS at 1080p), training in minutes. **Quality:** comparable to or better than NeRF on many scenes. **Cons:** large memory and storage (hundreds of MB), artifacts in sparse views, less compact, editing and anti-aliasing issues.

Transfer learning, few-shot and domain adaptation

DL03-190 What are Siamese networks, how are they trained, and where are they applied in computer vision?

A **Siamese network** has two (or more) branches with **shared weights** that embed two inputs, followed by a distance or similarity function. The goal is to learn an embedding where similar items are close. **Training:**

- **contrastive loss** on pairs: pull same-class pairs together and push different-class pairs at least a margin m apart ($L = y \cdot d^2 + (1 - y) \cdot \max(0, m - d)^2$);
- or **triplet loss** (anchor, positive, negative);
- or binary classification on the $|f(a) - f(b)|$ features.

Applications: face verification (DeepFace, FaceNet), signature and fingerprint verification, person and vehicle re-identification, one-shot classification, image retrieval and deduplication, object tracking (SiamFC, SiamRPN template matching), change detection in satellite imagery. It generalizes to **new identities without retraining**: just compare embeddings.

DL03-191 How do triplet loss and hard-negative mining work for metric learning, and why does mining strategy matter?

Triplet loss: for an anchor a, a positive p (same class) and a negative n (different class): $L = \max(0, \|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + \alpha)$. It requires the negative to be farther than the positive by margin α (FaceNet). **Why mining matters:** most random triplets are **easy** (already satisfy the margin, giving zero loss and no gradient), so training stalls. But the **hardest** negatives can be label noise or cause collapse (all embeddings equal) and unstable training. **Strategies:**

- **semi-hard** negatives (farther than the positive but within the margin; FaceNet), mined **online within large batches**;
- **batch-hard** (hardest positive and negative per anchor in the batch);
- distance-weighted sampling.

Alternatives that avoid mining: multi-similarity loss, N-pair or InfoNCE losses, and proxy-based or classification-margin losses (ArcFace), which are often more stable.

DL03-192 How do face recognition systems use margin-based losses such as ArcFace, and how do face verification and identification differ?

Margin-based softmax losses train face embeddings as a classification problem over training identities, with an **angular margin** for compactness:

- L2-normalize both the features and the class weights, so the logit = s·cos θ_y (θ_y is the angle to the class center, s the scale, e.g. 64).
- **ArcFace** adds an **additive angular margin m** (≈ 0.5) to the target angle: logit_y = s·cos(θ_y + m). The model must place a sample at least m radians closer to its own class center than the decision boundary, giving **tighter intra-class clusters and larger inter-class gaps** on the hypersphere. Related: SphereFace (multiplicative margin), CosFace (cos θ − m).
- At test time the classifier is discarded and the **normalized embedding** is used with cosine similarity.

Verification (1:1): "is this the same person as the claimed identity?" Compare two embeddings and threshold the similarity; evaluated with TAR at a fixed FAR (e.g. 1e-6), ROC. **Identification (1:N):** "who is this?" Search a gallery of N enrolled identities for the nearest match (plus open-set rejection if no match exceeds the threshold); evaluated with rank-1 accuracy and the FPIR/FNIR trade-off. It is harder as N grows.

DL03-193 How would you approach transfer learning when the target dataset is heavily imbalanced?

- **Two-stage fine-tuning:** first learn features (fine-tune the backbone with standard or mildly balanced sampling), then **retrain the classifier head with class-balanced sampling** (decoupled training).
- **Loss:** class-weighted cross-entropy, focal loss, class-balanced loss, LDAM with deferred re-weighting, logit adjustment.
- **Sampling:** oversample minority classes (with strong augmentation to avoid memorizing repeats); undersample the majority carefully.
- **Freeze more of the backbone** if minority classes have few examples, to avoid overfitting them; use a pretrained backbone with good general features.
- **Augment** minority classes more (Mixup or CutMix variants such as Remix that favor the minority label), synthetic generation.
- **Evaluate** with macro-F1, per-class recall, balanced accuracy and PR-AUC; **tune decision thresholds** per class on validation data.
- Consider anomaly detection or metric learning if a class is extremely rare.

DL03-194 How can you adapt a pre-trained model from one domain to a different but related domain?

- **Fine-tuning** on labeled target data (the most effective with even a few hundred labels): lower learning rate, unfreeze progressively, re-estimate BN statistics.
- **Unsupervised domain adaptation** (no target labels):
  - **feature alignment:** match source and target feature distributions (MMD, CORAL);
  - **adversarial** domain classifier with gradient reversal (DANN);
  - **self-training:** pseudo-label confident target samples and retrain (with consistency regularization, teacher–student);
  - **AdaBN:** recompute BatchNorm statistics on target data (simple and surprisingly effective);
  - **test-time adaptation** (TENT: entropy minimization).
- **Input-level alignment:** image translation (CycleGAN) to make source images look like the target, color and style normalization, physics-based augmentation.
- **Domain-specific self-supervised pretraining** on unlabeled target images (continued pretraining).
- Validate on a small labeled target set; monitor for negative transfer.

DL03-195 How can you measure how similar the source and target domains are before attempting transfer?

- **Feature-distribution distances:** embed both datasets with a pretrained network, then compute **FID / Fréchet distance**, **MMD**, CORAL distance (covariance difference) or Wasserstein distance between the feature distributions.
- **Proxy A-distance (domain classifier):** train a classifier to tell source from target. Accuracy near 50% means the domains are similar; near 100% means very different. d_A = 2(1 − 2·error).
- **Transferability estimators:** LogME, LEEP, NCE, H-score estimate how well pretrained features will fit target labels without full fine-tuning.
- **Low-level statistics:** color histograms, intensity distributions, resolution, object scale; quick visual inspection and t-SNE or UMAP plots of embeddings.
- **Empirical:** linear probe accuracy of source features on a small labeled target sample.

DL03-196 How can knowledge distillation be used as a form of transfer learning?

Knowledge distillation trains a **student** to mimic a **teacher**:

- **Response-based (Hinton):** minimize the KL divergence between temperature-softened teacher and student outputs, L = α·CE(y, p_s) + (1 − α)·T^2·KL(p_t^T || p_s^T). The soft targets ("dark knowledge") convey class similarities.
- **Feature-based:** match intermediate features or attention maps (FitNets, attention transfer); relation-based (pairwise similarity structure).

As transfer learning:

- transfer knowledge from a **large pretrained model** (or a model trained on a different, larger domain) into a **smaller or differently structured** student for deployment;
- **cross-modal distillation:** an RGB teacher supervises a depth, thermal or lidar student on paired unlabeled data;
- **unlabeled target data:** the teacher's soft labels on target-domain images act as pseudo-labels for adaptation (self-training and Noisy Student);
- distilling foundation models (CLIP, DINOv2, SAM) into task-specific efficient models.

DL03-197 How does adversarial training, as in domain-adversarial neural networks, help with unsupervised domain adaptation?

DANN (Domain-Adversarial Neural Network): a shared feature extractor feeds two heads:

1. a **label classifier**, trained on labeled source data;
2. a **domain classifier**, which predicts source versus target.

A **gradient reversal layer** between the features and the domain classifier multiplies the gradient by −λ during backprop. The domain classifier learns to distinguish domains, while the feature extractor is pushed to **fool it**, learning **domain-invariant features** that are also discriminative for the task. Theory (Ben-David et al.): target error ≤ source error + domain divergence + λ*, so minimizing divergence while keeping source error low bounds target error. **Caveats:**

- aligning marginal feature distributions can **misalign classes** (conditional shift), so class-conditional variants (CDAN) help;
- it fails if label distributions differ greatly;
- adversarial training is unstable.

DL03-198 What is meta-learning, and how does it relate to transfer learning?

Meta-learning ("learning to learn") trains across **many tasks** so the model can **adapt quickly** to a new task from a few examples.

- **Optimization-based (MAML):** learn an initialization from which a few gradient steps on a new task's support set give good performance. The outer loop optimizes the post-adaptation loss (second-order or first-order approximations such as Reptile).
- **Metric-based:** prototypical networks, matching networks, relation networks learn an embedding space where comparison works for new classes.
- **Model-based:** memory-augmented networks, black-box learners.

Relation to transfer learning: both reuse prior knowledge. Transfer learning pretrains on one big source task and fine-tunes once. Meta-learning explicitly optimizes for **fast adaptation** over a distribution of tasks, with episodic training matching the few-shot test setting. In practice, large-scale pretraining plus simple fine-tuning often matches meta-learning in few-shot benchmarks.

DL03-199 How would you extend a model trained to recognize car models so that it also recognizes trucks without forgetting cars?

This is **class-incremental (continual) learning**, where the risk is **catastrophic forgetting**.

- **Simplest robust option:** if the car data is still available, **retrain or fine-tune on cars plus trucks jointly**, with a new head covering all classes.
- **If old data is limited:**
  - **rehearsal / replay:** keep a small exemplar memory of car images (herding selection, as in iCaRL) and mix it with truck data;
  - **knowledge distillation (LwF):** keep the old model's outputs on new images as soft targets so car predictions do not drift;
  - **regularization (EWC):** penalize changes to weights important for cars (Fisher information);
  - **parameter isolation:** freeze the backbone and add new heads, adapters or a new branch for trucks;
  - **bias correction** for the classifier's preference toward new classes (BiC, weight aligning).
- **Evaluate** accuracy on old and new classes separately and overall (average incremental accuracy, forgetting measure).
- If trucks are a separate task, a separate head with a shared frozen backbone is simplest.

DL03-200 How would you transfer knowledge from a model trained on X-ray images to MRI images, and what could go wrong?

Approach:

1. Start from the X-ray model (or ideally a general medical or self-supervised model).
2. Adapt the input: MRI is often 3D (multi-slice), multi-sequence (T1, T2, FLAIR) and has a different intensity range, so modify the stem (channel mapping, 2.5D slices or a 3D-inflated network), use MRI-specific intensity normalization (z-score per volume, bias-field correction) and resampling.
3. **Fine-tune** most layers (large domain gap) with a small learning rate; re-estimate normalization statistics; apply **self-supervised pretraining on unlabeled MRI** first if available.
4. Augment realistically (rotations, intensity and bias-field variation, noise).
5. Validate with patient-level splits and radiologist review.

What can go wrong:

- **negative transfer:** X-ray features (projection images, bone contrast) do not match MRI tissue contrast, so performance can fall below training from scratch or MRI-pretrained baselines;
- different anatomy and pathology appearance, 2D versus 3D mismatch;
- scanner and protocol shift;
- overfitting with small MRI data;

- hidden label or prevalence differences;
- spurious cues (markers, text overlays) learned from X-ray.

DL03-201 What risks of inherited bias come with using pre-trained vision models?

- **Dataset bias:** ImageNet and web data over-represent certain geographies, skin tones, genders, Western objects and contexts, so models perform worse on under-represented groups (face recognition error disparities; household objects from low-income countries).
- **Stereotyped associations** (gender and occupation; CLIP linking demographics with offensive labels).
- **Shortcut features:** texture bias, background and context reliance.
- **Label noise and offensive categories** in source datasets (ImageNet person subtree issues).
- **Spurious correlations** transferred into downstream tasks (medical, hiring, policing) can cause unfair or harmful outcomes.
- **Privacy and consent** issues in scraped training data; licensing restrictions.

Mitigation: evaluate disaggregated metrics by subgroup, use balanced fine-tuning data, apply fairness constraints and audits, document models (model cards), choose pretraining data carefully, and keep humans in the loop for high-stakes use.

DL03-202 What theories explain why transfer learning works, and when does it give negative transfer?

Why transfer works:

- **Feature reuse:** early layers learn general, domain-agnostic features (edges, textures, Gabor-like filters), so lower layers transfer best (Yosinski et al.).
- **Better initialization:** pretrained weights land in a good basin of the loss landscape; fine-tuned models stay close to it (Neyshabur et al., "What is being transferred?"). Benefits come from both feature reuse and low-level statistics.
- **Theory:** domain-adaptation bounds (Ben-David), where target error ≤ source error + divergence + combined error; representation-learning views (shared low-dimensional structure across tasks).
- **Inductive bias and regularization** from large, diverse pretraining data.

Negative transfer occurs when:

- source and target domains or tasks are **dissimilar** (natural images versus spectrograms or medical images with different statistics);
- source features encode spurious or conflicting cues;
- the target dataset is large enough that the pretraining prior just constrains the model;
- the architecture or input mismatch is poorly adapted;
- there are optimization issues (too-high learning rate destroying features, BN statistics mismatch).

Data, robustness and evaluation in vision

DL03-203 What metrics would you use to evaluate a vision model for classification, detection and segmentation?

- **Classification:** accuracy and top-k accuracy; precision, recall, F1 (macro and micro); confusion matrix; ROC-AUC and PR-AUC (imbalanced data); log loss; calibration (ECE, reliability diagrams); per-class and per-slice metrics.
- **Detection: mAP** (COCO AP@[.5:.95], AP50, AP75, AP_S/M/L), average recall, precision and recall at an operating threshold, per-class AP, FPS and latency.
- **Segmentation: mIoU** and per-class IoU, Dice (medical), pixel accuracy (with caution), boundary F1 and HD95; for instance and panoptic tasks, mask AP and **PQ** (SQ, RQ).
- **Always also:** robustness (corruption and shift tests), fairness across subgroups, and efficiency (latency, memory).

DL03-204 How would you split an image dataset to avoid leakage from near-duplicates, the same patient or the same scene?

- **Deduplicate first:** detect exact and near-duplicates with perceptual hashing (pHash, dHash) or **embedding similarity** (CLIP or DINO features, cosine above a threshold), and put each duplicate cluster in **one** split.
- **Group-aware splitting:** split by **patient ID**, scene, video, camera, location, session, photographer or product, never by image (GroupKFold / StratifiedGroupKFold), so correlated images cannot leak.
- **Video and burst data:** split by whole video, not frame; for time-dependent data, split **by time** (train on the past, test on the future).
- **Keep class stratification** within groups where possible, and verify after splitting: run embedding nearest-neighbor search from test to train and inspect the closest pairs.
- Hold out entire sites or hospitals and devices to measure generalization to new sources.

DL03-205 How do you find and handle label noise in a large image dataset?

Finding noisy labels:

- **confident learning / cleanlab:** use out-of-fold predicted probabilities to estimate the joint noise distribution and flag likely mislabeled samples;
- **high-loss samples** (or samples whose loss stays high, area under the margin), disagreement between models or folds;
- **embedding-based outliers:** an image whose nearest neighbors carry a different label;
- **annotator disagreement**, cross-checks against metadata.

Handling:

- **relabel** flagged samples with expert review (prioritize by confidence) or remove the worst;
- **noise-robust training:** label smoothing, symmetric or generalized cross-entropy, bootstrapping loss, Co-teaching or DivideMix (train on small-loss samples, treat others as unlabeled), early stopping (networks memorize noise late);
- improve labeling guidelines and add consensus labeling for ambiguous classes;
- keep a **clean, expert-verified test set**.

DL03-206 How would you use active learning to decide which images to label next?

Loop:

1. Train on the current labeled set.
2. Score the unlabeled pool with an **acquisition function**.
3. Send the top-k images (a batch) to annotators.
4. Add them and retrain; repeat until the budget is spent or performance plateaus.

Acquisition strategies:

- **Uncertainty:** least confidence, margin (top-1 minus top-2), entropy; for detection, aggregate box-level uncertainties; Monte Carlo dropout or ensembles for epistemic uncertainty (BALD).
- **Diversity / representativeness:** core-set selection (k-center greedy in embedding space), clustering, to avoid redundant batches.
- **Hybrid:** BADGE (gradient embeddings plus k-means++), uncertainty within diverse clusters.

Practical points: start with a random or diverse seed set; account for class imbalance and the cold start; check for outliers and noise (uncertainty sampling picks ambiguous or garbage images); compare against random sampling, which is a strong baseline; consider labeling cost (boxes and masks cost more).

DL03-207 When does synthetic data or domain randomization help a vision model, and how do you close the sim-to-real gap?

Synthetic data helps when:

- real data is scarce, expensive or dangerous to collect (rare defects, accidents, medical rarities);

- **perfect labels** are needed (depth, segmentation, pose, optical flow);
- rare edge cases or controllable variation are needed;
- privacy blocks real data.

Sim-to-real gap: differences in appearance (textures, lighting, noise), content and distributions. **Closing it:**

- **Domain randomization:** heavily vary textures, lighting, camera parameters, backgrounds and distractors so that reality looks like "just another variation";
- **photorealistic rendering** (game engines, Omniverse) and accurate physical sensor models (noise, blur, lens distortion);
- **image translation:** GANs or diffusion models make synthetic images look real (CycleGAN), or style augmentation;
- **mix synthetic with a small amount of real data;** fine-tune on real data last;
- **domain adaptation** (adversarial feature alignment, AdaBN, pseudo-labeling on real data);
- validate on real held-out data and match the scene statistics (object scales, viewpoints).

DL03-208 What are adversarial examples for CNNs, and what defenses such as adversarial training work against them?

Adversarial examples are inputs with tiny, often imperceptible perturbations (e.g. $\|\delta\|_\infty \le 8/255$) crafted to cause misclassification. They exist because models are highly linear locally and rely on non-robust but predictive features. **Attacks:**

- **FGSM:** $x + \varepsilon \cdot \text{sign}(\nabla_x L)$;
- **PGD:** iterated FGSM with projection;
- C&W, AutoAttack;
- black-box transfer and query attacks;
- physical attacks (adversarial patches, stickers on stop signs).

Defenses:

- **Adversarial training** (Madry et al.): train on PGD examples, i.e. solve $\min_\theta \mathbb{E}[\max_{(\|\delta\| \le \varepsilon)} L(x + \delta, y)]$. It is the most reliable empirical defense; costly (≈ 7–10× training time) and reduces clean accuracy (TRADES balances the two).
- **Certified defenses:** randomized smoothing, interval bound propagation, which give provable radius guarantees.
- Input preprocessing and denoising, detection of adversarial inputs, ensembles. Many were broken by adaptive attacks ("obfuscated gradients").
- Evaluate with strong adaptive attacks (AutoAttack), not just FGSM.

DL03-209 What is shortcut learning in vision models, such as relying on backgrounds or textures, and how would you detect it?

Shortcut learning: the model exploits **spurious cues** that correlate with labels in the training data but not in deployment. **Examples:**

- cows recognized by green pasture backgrounds;
- pneumonia predicted from hospital-specific X-ray markers or portable-scanner artifacts;
- watermarks or text tags;
- **texture over shape** (ImageNet CNNs classify a cat-shaped object with elephant texture as an elephant).

Detection:

- **saliency and Grad-CAM** showing the model attends to background or markers;
- **counterfactual and controlled tests:** swap backgrounds (ImageNet-9 background challenge), cue-conflict stimuli (stylized ImageNet), mask the object or the background, crop out markers;
- evaluate on **out-of-distribution splits** (new hospitals, locations), where performance drops reveal shortcuts;
- slice analysis by metadata;

- training a model on the **suspected cue alone** (e.g. background only): high accuracy means a shortcut is available.

Mitigation: diverse and balanced data, augmentation that breaks the cue (Stylized ImageNet, background randomization), group-robust training (GroupDRO, JTT), removing cues at the source.

DL03-210 How would you measure a vision model's robustness to corruptions and distribution shift, for example with ImageNet-C-style tests?

- **Corruption benchmarks:** ImageNet-C / CIFAR-10-C (15 corruptions: noise, blur, weather, digital such as JPEG and pixelate, at 5 severities). Report **mean Corruption Error (mCE)**, normalized by a baseline (AlexNet), and **relative mCE** (degradation from clean accuracy). ImageNet-P measures prediction stability under small perturbations (flip rate).
- **Distribution-shift test sets:** ImageNet-V2 (re-collected), ImageNet-R (renditions), ImageNet-Sketch, ImageNet-A (natural adversarial examples), ObjectNet (unusual poses and backgrounds), WILDS (real-world domain shifts).
- **Domain-specific stress tests:** simulate your deployment conditions (camera noise, low light, compression, rain, motion blur, sensor dropout) and **measure the accuracy curve against severity**; slice by metadata (device, location, time).
- Also: calibration under shift, OOD detection AUROC, adversarial robustness (AutoAttack).
- Improve with augmentation (AugMix, DeepAugment), larger pretrained models and diverse training data.

DL03-211 Why are vision classifiers often miscalibrated, and how do you calibrate them?

Why miscalibrated: modern deep networks are **overconfident**: predicted confidence exceeds actual accuracy (Guo et al., 2017). Causes include high capacity and overfitting of the NLL (the network keeps increasing logit magnitude after accuracy saturates), weak weight decay and BN effects, and distribution shift, under which confidence remains high. **Measuring:** reliability diagrams; **Expected Calibration Error** (ECE: bin by confidence, weighted |accuracy − confidence|); NLL; Brier score; class-wise and adaptive ECE. **Calibrating:**

- **temperature scaling:** divide the logits by a scalar T > 1 fitted on validation NLL. Simple, keeps accuracy unchanged, very effective in-distribution;
- vector or matrix (Platt) scaling, isotonic regression, histogram binning, Dirichlet calibration;
- **training-time:** label smoothing, Mixup, focal loss, ensembles and MC dropout (better calibrated and better under shift);
- recalibrate after quantization and deployment changes; check calibration under shift too.

Applied vision system design and deployment

DL03-212 How would you design a computer vision system for automatic license plate recognition?

1. **Capture:** cameras with suitable resolution, shutter speed (motion blur), IR illumination for night; region-of-interest (lanes, gates).
2. **Vehicle and plate detection:** a real-time detector (YOLO or SSD) for plates (optionally detect vehicles first), robust to angle, size, lighting and occlusion.
3. **Rectification:** perspective correction or keypoint-based alignment of plate corners; handle two-line plates and different country formats.
4. **Recognition (OCR):** a CRNN with CTC, or an attention or transformer OCR on the plate crop (segmentation-free); a region-specific character set.
5. **Post-processing:** validate against country and format regex rules, character confusion handling (0/O, 1/I, 8/B); **temporal voting across video frames** with tracking (SORT) to choose the most confident reading.
6. **Data:** diverse real plates plus synthetic plate generation; augment blur, low light, rain, glare and viewing angles.

7. **Deployment:** edge inference (Jetson, NPU) for latency and bandwidth; send events to a backend database; privacy and retention compliance.
8. **Metrics:** detection recall, character and plate-level accuracy, end-to-end read rate, FPS, and failure analysis by condition.

DL03-213 What vision technologies does an autonomous vehicle perception stack use, and what neural network architecture would you choose for it?

Sensors: surround cameras, lidar, radar, ultrasonic sensors, GNSS and IMU. **Perception tasks:**

- 2D and 3D object detection (vehicles, pedestrians, cyclists);
- tracking and motion prediction;
- lane, road-boundary and drivable-area segmentation;
- traffic light and sign recognition;
- depth and 3D occupancy;
- free-space estimation;
- localization against HD maps (or mapless online mapping).

Architecture choice: a **multi-task, multi-sensor BEV network**:

- per-camera backbones (RegNet, EfficientNet or ConvNeXt) with an FPN;
- a **view transformation into a shared BEV space** (LSS or BEVFormer), fused with lidar (voxel or pillar encoders) and radar features (BEVFusion);
- **temporal fusion** across frames;
- **multiple heads:** 3D detection (CenterPoint or DETR-style), map segmentation, 3D occupancy, motion forecasting, sharing the backbone for efficiency.

Requirements: real-time (≥ 10–30 Hz) on automotive SoCs, redundancy and safety (separate fallback pipelines, uncertainty estimates), rigorous evaluation across weather, night and long-tail scenarios, and data engines that mine hard cases from the fleet. Trends: end-to-end and occupancy networks.

DL03-214 How would you design a deep learning system for medical image diagnosis?

1. **Problem definition with clinicians:** task (classification, detection, segmentation), clinical endpoint, intended use and population, acceptable sensitivity and specificity at the operating point.
2. **Data:** multi-site, multi-scanner datasets; **expert labels** (multiple readers, adjudication, inter-rater agreement); DICOM handling, de-identification; **patient-level splits**; external test set from unseen hospitals.
3. **Preprocessing:** modality-specific normalization (CT windowing, MRI z-score), resampling to isotropic spacing, 2D, 2.5D or 3D patches.
4. **Model:** nnU-Net or U-Net-family networks for segmentation, CNN or ViT classifiers, pretrained on medical data (RadImageNet, self-supervised); class-imbalance losses (Dice + focal); appropriate augmentation.
5. **Evaluation:** AUC, sensitivity and specificity with CIs, Dice and HD95, calibration, subgroup analysis (age, sex, scanner), external validation, reader studies comparing against and with radiologists.
6. **Explainability and safety:** saliency maps, uncertainty estimates, OOD detection, human-in-the-loop workflow (triage or second reader, not autonomous).
7. **Deployment and regulation:** PACS integration, monitoring for drift (new scanners), FDA or CE approval, privacy (HIPAA, GDPR), post-market surveillance.

DL03-215 How would you design a real-time object detection system for video streams?

1. **Ingestion:** decode RTSP or streams with hardware decoders (NVDEC); frame skipping or adaptive FPS; batch frames from multiple streams.
2. **Detector:** a real-time model (YOLO or RT-DETR) at an input size chosen for the speed and accuracy target; TensorRT FP16 or INT8; dynamic batching.
3. **Tracking:** ByteTrack or DeepSORT, for stable IDs, smoothing and fewer per-frame detections (detect every N frames and track in between).

4. **Post-processing:** NMS (GPU-based), zone rules, event logic (counting, intrusion).
5. **Pipeline:** an asynchronous producer–consumer design (DeepStream or GStreamer), with GPU-resident data to avoid copies and backpressure handling; latency budget per stage.
6. **Scaling:** edge devices near the cameras for latency and bandwidth, or a central GPU cluster; horizontal scaling per stream count; graceful degradation (lower FPS or resolution under load).
7. **Monitoring:** throughput, end-to-end latency (p95), dropped frames, detection quality on sampled frames, drift; retraining with hard examples from production.

DL03-216 How would you build a product recognition and tracking system for retail stores?

- **Recognition:** products are many (10k+ SKUs), fine-grained, frequently changing, with packaging updates. Use a **detector** for "product" or category-level boxes, then **embedding-based retrieval** (metric learning with ArcFace or contrastive loss, on product crops) against a reference gallery, so new SKUs are added by enrolling images without retraining. Use OCR on labels and barcodes when visible.
- **Data:** studio reference images plus in-store photos; synthetic shelf rendering; domain adaptation from studio to shelf images.
- **Shelf analytics:** planogram compliance, out-of-stock detection (empty-space detection), price-tag reading (OCR) and matching.
- **Tracking (checkout-free or shopper analytics):** multi-camera person tracking (re-ID), hand–product interaction detection (pick or put events), sensor fusion with weight shelves.
- **Deployment:** edge inference in-store, periodic shelf scans (robots or fixed cameras), privacy protection (no face storage).
- **Metrics:** top-1/top-5 retrieval accuracy, detection mAP, OOS detection precision and recall, latency.

DL03-217 How would you use CNNs to monitor and classify satellite imagery?

- **Data specifics:** multispectral or hyperspectral bands (beyond RGB: near-infrared, short-wave infrared), very large tiles (tile into 256–512 px patches with overlap), varying resolution (GSD), cloud cover, seasonal and atmospheric variation, georeferencing.
- **Preprocessing:** atmospheric correction, cloud masking, per-band normalization, co-registration; use indices (NDVI) as extra channels.
- **Models:**
  - scene classification with CNNs or ViTs (adapt the first conv to N bands);
  - semantic segmentation (U-Net or DeepLab) for land cover, buildings and roads;
  - detection for ships, planes and vehicles, with **rotated bounding boxes**;
  - **change detection** with Siamese or temporal models over image time series.
- **Pretraining:** remote-sensing foundation models (SatMAE, Prithvi, SSL4EO) rather than ImageNet.
- **Augmentation:** rotations and flips (no canonical orientation), radiometric jitter.
- **Evaluate** with spatially separated splits (avoid geographic leakage), per-region metrics.
- **Monitoring use-cases:** deforestation, crops, disasters, urban growth; deploy as batch pipelines over new acquisitions.

DL03-218 How do augmented reality applications use computer vision for tracking, mapping and scene understanding?

- **Tracking and mapping (SLAM / VIO):** detect and track visual features (ORB, FAST, learned features) and fuse them with the IMU (visual-inertial odometry) to estimate the **6-DoF device pose** in real time, while building a sparse or dense map of the environment (ARKit, ARCore). Loop closure and relocalization correct drift.
- **Scene understanding:**
  - plane detection (floors, tables) for placing objects;
  - **depth estimation** (LiDAR, stereo, or monocular networks) for occlusion and physics;

  - **semantic segmentation** and object detection (people, furniture) for occlusion masks and interactions;
  - light estimation for realistic rendering.
- **Image and marker tracking:** recognize known images or objects and track their pose (homography or PnP).
- **Human-centric:** face meshes, hand tracking, body pose for filters and interaction.
- **Constraints:** very low latency (< 20 ms motion-to-photon), mobile power and compute, robustness to motion blur and low texture.

DL03-219 How would you design a visual search system that finds similar products from a user's photo?

1. **Query processing:** detect and crop the product in the user's photo (a detector or SAM-assisted crop), handle multiple items, normalize.
2. **Embedding model:** a CNN or ViT (fine-tuned CLIP, DINOv2 or a domain-trained model) trained with **metric learning** (contrastive or triplet, ArcFace on product IDs, or a multimodal image–text contrastive loss) so that the same or similar products cluster despite lighting, background and angle; train on street-to-shop pairs (user photos versus catalog images).
3. **Index:** embed all catalog images offline; store them in an **approximate nearest neighbor** index (FAISS, HNSW or IVF-PQ) for millisecond search at scale; update incrementally for new products.
4. **Retrieval and ranking:** top-k ANN candidates, then re-rank with a finer model and business signals (availability, price, popularity), category filters and attribute matching (color, pattern).
5. **Evaluation:** recall@k and mAP on street-to-shop test pairs, plus online CTR and conversion.
6. **Operations:** latency budget, embedding versioning (re-index on model updates), deduplication, monitoring.

DL03-220 How would you design a defect detection system for a factory line where defective samples are very rare?

With very few defects, frame it mainly as **anomaly detection** (learn "normal"):

- **Data capture:** controlled imaging (fixed lighting, camera, triggering) to reduce nuisance variation; collect many normal samples.
- **Unsupervised anomaly detection:** methods trained only on good parts:
  - **PatchCore** (memory bank of pretrained patch features with nearest-neighbor distance);
  - PaDiM (Gaussian per patch location);
  - student–teacher, reconstruction-based (autoencoders, diffusion) or normalizing flows (FastFlow).

  They output an image-level score and a **pixel-level anomaly map** to localize defects.
- **Use the few defects you have** for validation and threshold setting, and for supervised or semi-supervised fine-tuning; **synthesize defects** (CutPaste, DRAEM-style synthetic anomalies, rendered scratches) for augmentation.
- **Thresholding:** choose an operating point that meets the miss-rate requirement (high recall); route borderline cases to human inspection.
- **Evaluate:** image AUROC, pixel AUROC and PRO, recall at a fixed false-positive rate; benchmark on MVTec AD-style splits.
- **Deployment:** inference within line takt time on edge GPUs or industrial PCs; monitoring for drift (lighting, new product variants); feedback loop to label new defect types.

DL03-221 How would you count people in dense crowd images, and why does detection-based counting break down there?

Why detection breaks down:

- extreme density and **heavy occlusion**: heads overlap and only fragments are visible;
- **tiny objects** (a few pixels per head) and large **scale variation** from perspective;
- NMS suppresses true neighbors;
- box annotations are impractical.

Density-map regression instead:

- annotate only **head points**, convolve each with a (geometry-adaptive) Gaussian kernel to get a **density map** whose **integral equals the count**;
- train a CNN (MCNN multi-column, CSRNet with dilated convs, or transformer-based) to regress the density map with a pixel-wise L2 loss, or a Bayesian or distribution-matching loss (DM-Count);
- the count is the sum of the predicted map, which also shows where people are.

Alternatives: point-based localization (P2PNet) to get counts plus locations. Handle perspective with multi-scale features. **Evaluate** with MAE and RMSE of counts (ShanghaiTech, UCF-QNRF), and check robustness to lighting and camera angle.

DL03-222 How do you export a vision model with ONNX or TensorRT, and what accuracy checks do you run after INT8 calibration?

Export:

1. Set the model to eval mode; **export to ONNX** (torch.onnx.export) with a fixed or dynamic input shape and the right opset; validate with ONNX Runtime and check for unsupported ops.
2. Build a **TensorRT engine** (trtexec or the Python API) for the target GPU, with FP16 and INT8 enabled, optimization profiles for dynamic shapes and layer fusion. INT8 needs **calibration** with a representative dataset (entropy or minmax calibrators, ~500–1000 diverse images in production-like conditions), or a QAT model with Q/DQ nodes.

Accuracy checks after INT8:

- compare outputs to the FP32 PyTorch model **layer-wise and end-to-end**: cosine similarity and max abs error of logits;
- task metrics on the full validation set: **top-1 and mAP drop** (typically < 1% is acceptable);
- **per-class and per-slice metrics** (small objects, rare classes, low-light images), where quantization errors concentrate;
- confidence calibration and **threshold re-tuning**, since score distributions shift;
- identical **preprocessing and post-processing** (NMS, decoding) in the deployed pipeline;
- sensitivity analysis: keep sensitive layers in FP16 (mixed precision) if accuracy drops;
- latency and throughput on the device.

DL03-223 Your detector must run at 30 FPS on an edge device. How do you budget latency across preprocessing, model and post-processing?

At 30 FPS the total budget is **≈ 33 ms per frame** (less if other workloads share the device; target ≤ 25 ms p95 for headroom).

1. **Measure each stage** on the device: capture and decode, preprocessing (resize, normalize, layout conversion), model inference, post-processing (decode boxes, NMS, tracking), output and rendering.
2. **Typical budget:** decode and preprocessing 3–5 ms (do them on GPU or ISP with hardware resize, avoid CPU copies), inference 15–20 ms, post-processing 2–5 ms, slack 3–5 ms.
3. **Optimize the model:** smaller backbone or input size, TensorRT or NPU compilation, INT8 or FP16, structured pruning, operator fusion.
4. **Optimize the pipeline: pipelining and parallelism** (preprocess frame t+1 while inferring frame t; overall throughput is limited by the slowest stage, while latency is the sum), zero-copy buffers, efficient or class-agnostic NMS, top-k pre-filtering before NMS.
5. **Reduce work:** run the detector every N frames and track in between; region-of-interest cropping.
6. **Validate:** p50, p95 and p99 latency, thermal throttling over long runs, power limits, and accuracy at the chosen settings.

CNN arithmetic and feature extraction

DL03-224 VGG-16 has about 138 million parameters. How much storage does it need in fp32, and in fp16?

- **FP32:** 4 bytes per parameter, so $138 \times 10^6 \times 4 \approx$ **552 MB** (≈ 527 MiB). The torchvision VGG-16 checkpoint is ≈ 528 MiB.
- **FP16:** 2 bytes per parameter, so ≈ **276 MB** (≈ 263 MiB).
- INT8 would be ≈ 138 MB.

Most of this sits in the first FC layer (25,088 × 4,096 ≈ 103M parameters). This is why replacing FC heads with global average pooling shrinks models dramatically. Note that runtime memory also includes activations (and, during training, gradients and optimizer states).

DL03-225 Describe the VGG-19 architecture: the number of convolutional and fully connected layers, kernel sizes, pooling, input size and preprocessing.

- **19 weight layers: 16 convolutional + 3 fully connected.**
- **Input:** 224×224×3 RGB; preprocessing subtracts the **mean RGB value** of the training set (per-channel mean subtraction).
- **Convolutions:** all **3×3**, **stride 1, padding 1** (spatial size preserved).
- **5 blocks** of convs with 2, 2, 4, 4, 4 conv layers and **64, 128, 256, 512, 512** channels, each block followed by **2×2 max pooling with stride 2** (5 pools total, 224 → 7).
- **ReLU** after every conv and FC (no batch norm in the original).
- **FC layers:** 4096 → 4096 → 1000, with **dropout (0.5)** between the first FCs, and **softmax** at the end.
- ≈ **144M parameters**, ≈ 19.6 GFLOPs per image.

DL03-226 Which layers of a pretrained CNN make the best fixed feature extractors, and why are the layers closest to the output not always the best choice?

- **Early layers** (conv1–conv2): generic edges and colors; transferable but not discriminative enough on their own.
- **Middle to late conv layers** (e.g. conv4 or conv5, or the last conv block before pooling): a good balance of semantic content and generality; often the **best for transfer**, especially to domains different from ImageNet. Aggregate them with global average or max pooling (or GeM).
- **Penultimate FC layers** (fc6/fc7 in VGG; the pooled 2048-d vector in ResNet): semantic and compact, very good when the target domain resembles ImageNet.
- **Layers closest to the output** (logits or fc8) are **specialized to the source task's 1000 classes**: features become task-specific, invariant to details the target task may need (texture, fine structure, position), and lower-dimensional. They transfer worse to dissimilar tasks (medical, texture, fine-grained).

Rule: the more different the target domain, the earlier the layer. Choose by validation (linear probe per layer), or concatenate features from multiple layers.

DL03-227 How can you build an ensemble of fixed feature extractors from different layers or different pretrained CNNs?

- **Multi-layer features from one network:** extract features from several layers (e.g. conv4, conv5 pooled with GAP or GeM, plus fc7), L2-normalize each, concatenate (optionally reduce with PCA), and train a linear classifier or SVM on top. This captures both low-level texture and high-level semantics (hypercolumns).
- **Multiple pretrained networks:** extract embeddings from different architectures or pretraining schemes (ResNet, EfficientNet, ViT, CLIP, DINOv2), which bring complementary errors. Then either:
  - **early fusion:** concatenate the normalized embeddings, then train one classifier; or
  - **late fusion:** train a separate classifier per extractor and average or vote their probabilities, or **stack** them with a meta-classifier trained on out-of-fold predictions.
- **Tips:** normalize per extractor so no single high-dimensional feature dominates; regularize (high-dimensional concatenation can overfit small data); measure each member's contribution; consider the inference cost of

running several backbones.

DL03-228 What is the difference between convolution and cross-correlation, and which one do deep learning libraries actually compute?

- **Cross-correlation:** $(f \star k)[i, j] = \Sigma_m \Sigma_n f[i + m, j + n] \cdot k[m, n]$. Slide the kernel as-is.
- **Convolution:** $(f * k)[i, j] = \Sigma_m \Sigma_n f[i - m, j - n] \cdot k[m, n]$. Equivalent to cross-correlation with the kernel **flipped by 180°**.

Convolution is commutative and associative (useful in signal processing, where filters are composed); correlation is a similarity or template-matching measure. **Deep learning libraries** (PyTorch Conv2d, TensorFlow, cuDNN) actually compute **cross-correlation** but call it convolution. Since kernels are learned, the flip does not matter: the network would just learn the flipped kernel. The flip appears in the **backward pass** (the gradient with respect to the input uses the rotated kernel).

DL03-229 How many multiplications does convolving a w×h image with a K×K kernel take, and how does a separable kernel such as a Gaussian reduce that cost?

For a w×h single-channel image and a K×K kernel (same-size output):

- **Direct convolution: $w \cdot h \cdot K^2$** multiplications and $w \cdot h \cdot (K^2 - 1)$ additions per channel.
- **Separable kernel** (rank 1: $K = u \cdot v^T$, e.g. Gaussian, box, Sobel): two 1D passes (rows with a K×1 kernel, then columns with a 1×K kernel) need **$2 \cdot w \cdot h \cdot K$** multiplications, a K/2 speed-up (e.g. 15×15: 225 versus 30 multiplications per pixel, 7.5× faster).

The 2D Gaussian factorizes as $G(x, y) = G(x) \cdot G(y)$. Very large kernels can also use **FFT** convolution, $O(wh \cdot \log(wh))$ regardless of K. For $C_{in} \rightarrow C_{out}$ channels, multiply by $C_{in} \cdot C_{out}$.

DL03-230 Why do you L2-normalize CNN image embeddings before similarity search or face matching?

- **Removes magnitude effects:** embedding norms vary with image contrast, size or "confidence" rather than identity or content. Normalizing puts all vectors on the unit hypersphere so comparisons depend only on **direction** (semantic similarity).
- **Cosine similarity becomes a dot product:** fast matrix-multiply and MIPS/ANN search; Euclidean distance on normalized vectors is monotonic in cosine ($\|a - b\|^2 = 2 - 2 \cdot \cos$), so any metric index works consistently.
- **Consistent thresholds:** similarity is bounded in $[-1, 1]$, so a single verification threshold works across all pairs (face matching).
- **Matches training:** metric-learning losses (ArcFace, CosFace, contrastive or InfoNCE with cosine similarity) are trained on normalized embeddings, so inference must normalize too.
- **Stability:** reduces the influence of outlier dimensions and scale drift across model versions or domains.

DL03-231 What output shapes does the VGG-11 feature extractor produce for 224×224, 512×512 and 704×1024 inputs, and why can it accept any input size?

VGG-11's feature extractor has 8 conv layers (3×3, padding 1, so spatial size is preserved), each followed by ReLU, and **5 max-pool layers** (2×2, stride 2), so the spatial size is divided by 32 (floored at each pool). The final channel count is 512. Its **last layer is a MaxPool2d.**

- **224×224 → [1, 512, 7, 7]** (224/32 = 7).
- **512×512 → [1, 512, 16, 16].**
- **704×1024 → [1, 512, 22, 32]** (704/32 = 22, 1024/32 = 32).

It accepts any input size because it contains **only convolutions, ReLUs and pooling**, which slide over whatever spatial extent they get (no layer has a fixed input dimension). Only the FC classifier needs a fixed 7×7×512 input, which torchvision handles with AdaptiveAvgPool2d((7, 7)). Inputs smaller than 32×32 would collapse to zero size.

DL03-232 For a 224×224×3 input, a 7×7 convolution with 32 filters and padding 2, followed by two 2×2 max-pools with stride 2, what is the output shape?

1. **Conv 7×7, 32 filters, padding 2, stride 1:** H_out = (224 + 2·2 − 7)/1 + 1 = **222**, giving **32 × 222 × 222**. BatchNorm, if present, does not change the shape.
2. **MaxPool 2×2, stride 2:** ⌊222/2⌋ = **111**, giving 32 × 111 × 111.
3. **MaxPool 2×2, stride 2:** ⌊111/2⌋ = **55** (floor), giving **32 × 55 × 55**.

Output tensor: **[N, 32, 55, 55]**; flattened, 32 × 55 × 55 = **96,800** features per image. The conv has 7·7·3·32 + 32 = 4,736 parameters.